Two-Scale Stochastic Simulation of Epidemic Dynamics in Community Networks

Yulian Michael

Version v4 — June 26, 2026

Abstract

We investigate continuous-time stochastic simulation of epidemic spreading on community-structured networks in settings where local contact data and inter-community mobility information are available at different levels of resolution. The proposed two-scale simulation framework preserves within-community dynamics as exact event-driven continuous-time Markov chain processes, implemented via Gillespie simulation, while replacing explicit cross-community edges with macro-level importation hazards that depend on aggregate community states. We first formulate the full-network continuous-time Markov chain as the reference process and then identify the precise sources of approximation introduced by the two-scale representation: the abstraction of inter-community transmission rates, the allocation of imported infections to recipient nodes, and the numerical synchronization between local and global layers. Our analysis shows that a macro-level hazard may correctly represent the conditional mean of the underlying microscopic bridge hazard while still inducing bias in infection arrival-time distributions, because event timing depends on accumulated hazards and survival probabilities rather than instantaneous rates alone. Using clique–bridge–leaf and chain-of-cliques benchmark networks, we isolate this interface error and characterize regimes in which it remains small, accumulates over time, or becomes pronounced due to narrow boundary bottlenecks. These results demonstrate that local Gillespie exactness and global network-level accuracy are distinct properties. The two-scale method is effective when boundary exposure is representative of the broader community state, but it requires calibration or bridge-aware corrections when inter-community transmission is governed by a small subset of boundary nodes.

1 Introduction

Epidemic spreading in large structured populations is governed by the interaction between contact-network heterogeneity, local stochasticity, and spatial coupling between communities. Network epidemiology has shown that individual-level contact structure can affect epidemic thresholds, invasion probabilities, outbreak sizes, extinction, and the timing of transmission events [?, ?, ?, ?]. At larger spatial scales, spread between cities, regions, or countries is often mediated by mobility processes, including commuting, travel, and transportation flows. These processes are naturally represented through metapopulation and mobility-network models, in which local populations are coupled by weighted mobility or transportation layers [Colizza et al., 2007, Colizza and Vespignani, 2008, ?, Balcan et al., 2010, Barbosa et al., 2018]. Recent data-driven mobility models for COVID-19 further illustrate the value of combining epidemic dynamics with empirically estimated flows between locations [?]. For applications in health informatics and bioinformatics, this creates

a practical need for simulation methods that retain continuous-time stochastic dynamics where local contact information is available, while representing long-range coupling at the coarser resolution at which mobility data are typically observed.

This requirement creates both a modeling and a computational challenge. A full continuous-time Markov chain (CTMC) on a microscopic contact network provides a natural reference process and can be simulated exactly using Gillespie-type methods [Gillespie, 1977, Anderson, 2007]. However, at regional or national scale, the edge-level contact structure required by such a model is rarely observed, and the number of event channels can be very large. Algorithmic advances such as temporal Gillespie methods and recent sparse-network Gillespie implementations improve the efficiency of exact or event-driven stochastic simulation, but they do not remove the need to specify the relevant microscopic transmission channels [Vestergaard and Génois, 2015, Kuryliak et al., 2025]. By contrast, aggregate flows between locations are often easier to estimate from commuting matrices, transportation records, mobile-phone summaries, administrative mobility data, or calibrated agent-based and metapopulation models [Ajelli et al., 2010, Kerr et al., 2021, **?**]. This asymmetry between local contact information and inter-community mobility information motivates a two-scale representation in which explicit stochastic network dynamics are retained within communities, while aggregate stochastic coupling is used between communities.

The potential computational gain can be substantial. Suppose there are k communities, each with n individuals, so that the full system has $N = kn$ nodes. In a dense microscopic representation, the number of possible node-level transmission channels scales as $O(N^2) = O(k^2n^2)$. In the two-scale representation, explicit contacts are retained only within communities, giving $O(kn^2)$ local channels, while inter-community transmission is represented by $O(k^2)$ aggregate coupling terms. The resulting bookkeeping therefore scales as $O(kn^2 + k^2)$ rather than $O(k^2n^2)$, corresponding to an idealized reduction factor of approximately $kn^2/(n^2 + k)$. This factor is of order $O(k)$ when $n^2 \gg k$, and of order $O(n^2)$ when $k \gg n^2$. This illustrates the computational appeal of the approach, while also motivating the central accuracy question: how much information is lost when explicit bridge edges and boundary-node heterogeneity are replaced by aggregate importation hazards?

Existing approaches address important parts of this problem, but not the interface considered here. Metapopulation and mobility-network epidemic models are scalable and compatible with aggregate data, but their within-location mixing assumptions can obscure rare-event timing and the role of specific boundary nodes [Colizza and Vespignani, 2008, Balcan et al., 2010, Ajelli et al., 2010]. Individual-based and agent-based models provide greater local detail and have been used for realistic urban or national-scale epidemic simulations, but they require detailed population and contact data and can be computationally demanding [**?**, Kerr et al., 2021]. Network CTMCs, Gillespie simulation, random-time-change constructions, temporal Gillespie methods, and efficient Gillespie algorithms on complex networks provide exact or event-driven descriptions of state-dependent stochastic dynamics on fixed or time-varying networks [Gillespie, 1977, Anderson, 2007, Vestergaard and Génois, 2015, Kuryliak et al., 2025]. The gap addressed in this paper is the construction and evaluation of an event-driven two-scale simulator that preserves exact local CTMC dynamics while replacing only the inter-community layer with a macro-level stochastic mechanism.

The main difficulty lies at the interface between the two scales. In the full microscopic CTMC, transmission from community i to community j depends on the states of specific cross-community edges, boundary nodes, and susceptible targets. A macro hazard based only on community-level susceptible and infectious fractions discards this informa-

tion. Such a hazard may still represent the correct average transmission pressure, but first-arrival distributions depend on accumulated hazards and survival probabilities, not only on instantaneous mean rates. Consequently, aggregate macro hazards can introduce timing bias when export is controlled by a small bridge set or by another narrow boundary bottleneck.

This paper studies a two-scale stochastic simulator designed to make this approximation explicit. The population is partitioned into communities. The micro layer simulates the CTMC inside each community exactly using Gillespie-type event-driven dynamics on the induced local graph. The macro layer replaces explicit cross-community infection edges by stochastic import hazards on a directed weighted graph of communities. Macro events are generated through cumulative hazard accumulation, so inter-community timing remains continuous rather than being imposed by fixed time steps. The full-network CTMC is used throughout as the reference process.

The contributions of the paper are fourfold:

1. We formulate a two-scale continuous-time stochastic simulation architecture with exact local CTMC/Gillespie engines and macro-level stochastic import hazards between communities.

2. We define the full microscopic CTMC reference model and show that, conditional on the import times and target nodes, the micro-layer dynamics are exact; the approximation enters through the generation and placement of imports.

3. We decompose the resulting interface error into rate abstraction error, import placement error, and synchronization or hazard-integration error.

4. We analyze clique–bridge–leaf and chain-of-cliques benchmarks to show how loss of bridge-node information can bias inter-community arrival times and how such local timing errors can accumulate across a community network.

The remainder of the paper is organized as follows. Section 2 introduces the notation, defines the full-network CTMC reference process, and specifies the two-scale simulator, including the micro layer, the macro layer, cumulative-hazard event timing, and synchronization between layers. Section 3 develops the accuracy framework and decomposes the interface error into rate abstraction, import placement, and synchronization components. It then analyzes the clique–bridge–leaf benchmark, the mean-field interpretation of bridge hazards, a numerical clique–bridge–leaf benchmark, and the chain-of-cliques benchmark to show how local interface errors can propagate through a community network. Section 4 summarizes the conclusions and outlines extensions, including bridge-aware hazards, improved import placement, calibration against microscopic reference simulations, and larger-scale computational experiments.

2 Mathematical Model and Architecture of the Two-Scale Stochastic Simulator

We use the following notation consistently. The full microscopic state is $X(t)$, while $X_i(t)$ is its restriction to community V_i. Capital letters $S_i(t)$, $I_i(t)$, and $R_i(t)$ denote sets of susceptible, infectious, and removed nodes in community i. Lowercase letters $s_i(t)$, $i_i(t)$, and $r_i(t)$ denote their cardinalities. Barred quantities such as $\bar{s}_i(t) = s_i(t)/N_i$ and $\bar{i}_i(t) = i_i(t)/N_i$ denote normalized community fractions. The notation $i_i(t)$ means "the

number of infectious nodes in community i"; the first i is the state label and the subscript i is the community index.

The symbols $a^\star_{ij}$ and λ_{ij} are deliberately kept distinct. The quantity $a^\star_{ij}(x)$ is the exact inter-community infection intensity from community i to community j in the full microscopic CTMC at state x. The quantity $\lambda_{ij}(t)$ is the macro-level approximation used by the two-scale simulator. Similarly, $\pi^\star_{ij}$ denotes the exact edge-conditioned target distribution in the full network, whereas Π_{ij} denotes the import-placement rule chosen by the two-scale simulator.

Symbol	Meaning
$G = (V, E)$	full microscopic contact graph
$V = \bigsqcup_{i=1}^{M} V_i$	fixed partition into communities
$N_i = \lvert V_i \rvert$	size of community i
E_{ii}	within-community edges in V_i
E_{ij}	oriented edges from V_i to V_j, $i \neq j$
S_i, I_i, R_i	node sets in community i
s_i, i_i, r_i	corresponding counts
$\bar{s}_i, \bar{i}_i, \bar{r}_i$	corresponding fractions
$a^\star_{ij}$	exact microscopic inter-community intensity
λ_{ij}	macro inter-community hazard approximation
Θ	total macro hazard, $\Theta = \sum_{i \neq j} \lambda_{ij}$
$\pi^\star_{ij}$	exact edge-conditioned target distribution
Π_{ij}	two-scale import-placement distribution

2.1 Full-Network CTMC Reference Model

Before introducing the two-scale approximation, we define the exact microscopic process on the full network. This process is the reference model against which all later approximation errors are measured.

Let $G = (V, E)$ be the full microscopic contact network. Throughout the generator below, E is interpreted as the set of oriented transmission channels; an undirected contact is represented by the two oriented channels whenever transmission is possible in both directions. Each node has state in $\{S, I, R\}$, and the node set is partitioned into communities

$$V = \bigsqcup_{i=1}^{M} V_i.$$

For a configuration $x \in \{S, I, R\}^V$, the infection propensity along an oriented edge $(u, v) \in E$ is

$$a_{u \to v}(x) = \beta_{uv}\, \mathbf{1}\{x_u = I,\ x_v = S\}.$$

The recovery propensity of node $u \in V$ is

$$a^{\mathrm{rec}}_u(x) = \gamma_u\, \mathbf{1}\{x_u = I\}.$$

Here β_{uv} is the edge-level transmission rate and γ_u is the node-level recovery rate. For a node v, let $x^{v:I}$ denote the configuration obtained from x by changing node v to state I. Similarly, $x^{u:R}$ denotes the configuration obtained by changing node u to state R.

The full-network CTMC is the process with generator

$$\begin{aligned}(\mathcal{L}f)(x) = &\sum_{(u,v) \in E} \beta_{uv}\, \mathbf{1}\{x_u = I,\ x_v = S\}\big[f(x^{v:I}) - f(x)\big] \\ &+ \sum_{u \in V} \gamma_u\, \mathbf{1}\{x_u = I\}\big[f(x^{u:R}) - f(x)\big],\end{aligned}$$

for bounded test functions f. This generator gives the exact full-network Gillespie/CTMC reference process. The two-scale simulator introduced below is therefore not defined in isolation: it is an approximation of this reference process, obtained by preserving local within-community CTMC dynamics while replacing the full collection of cross-community infection channels by macro-level import mechanisms.

2.2 Two-Scale Simulator

The two-scale simulator is defined on the fixed community partition

$$V = \bigsqcup_{i=1}^{M} V_i, \qquad N_i = |V_i|.$$

For each community i, let E_{ii} denote the within-community edges and let

$$E_{ij} = \{(u, v) \in E : \ u \in V_i, \ v \in V_j\}, \qquad i \neq j,$$

denote the oriented inter-community edges from community i to community j in the full reference network.

At time t, the node-level state in community i is summarized by the sets

$$\begin{aligned} S_i(t) &= \{v \in V_i : X_v(t) = S\}, \\ I_i(t) &= \{v \in V_i : X_v(t) = I\}, \\ R_i(t) &= \{v \in V_i : X_v(t) = R\}. \end{aligned}$$

We distinguish these sets from their cardinalities,

$$s_i(t) = |S_i(t)|, \qquad i_i(t) = |I_i(t)|, \qquad r_i(t) = |R_i(t)|,$$

and from the normalized macro summaries

$$\bar{s}_i(t) = \frac{s_i(t)}{N_i}, \qquad \bar{i}_i(t) = \frac{i_i(t)}{N_i}, \qquad \bar{r}_i(t) = \frac{r_i(t)}{N_i}.$$

Thus $S_i(t), I_i(t), R_i(t)$ are node sets, $s_i(t), i_i(t), r_i(t)$ are counts, and $\bar{s}_i(t), \bar{i}_i(t), \bar{r}_i(t)$ are fractions.

Each community has a local micro CTMC engine running on the induced graph $G_i = (V_i, E_{ii})$. Between external imports, this engine simulates exactly the within-community infection and recovery channels. The macro layer observes only the community summaries, not the full boundary-node configuration. For each ordered pair $i \neq j$, it assigns the effective inter-community hazard

$$\lambda_{ij}(t) = \beta_{\text{out}} \, W_{ij} \, \bar{i}_i(t) \, \bar{s}_j(t),$$

where W_{ij} is the aggregate coupling weight from community i to community j, and β_{out} is the macro-level transmission-rate parameter.

Let

$$\Theta(t) = \sum_{i \neq j} \lambda_{ij}(t)$$

be the total macro hazard. When a macro event occurs at time t and $\Theta(t) > 0$, the ordered source–destination pair is selected according to

$$\mathbb{P}\{(i, j) \text{ is selected} \mid \text{macro event at } t\} = \frac{\lambda_{ij}(t)}{\Theta(t)}.$$

After selecting $i \to j$, the simulator chooses a target node in the destination community using an import placement rule

$$\Pi_{ij}(\cdot \mid t) \quad \text{on} \quad S_j(t).$$

For example, a simple rule is uniform placement over susceptible nodes,

$$\Pi_{ij}(v \mid t) = \frac{1}{s_j(t)}, \qquad v \in S_j(t),$$

when $s_j(t) > 0$. More structured rules may use degree, boundary information, or an edge-conditioned oracle distribution. Once a target $v \in S_j(t)$ is selected, the destination microstate is updated by the imported jump $v : S \to I$, and the local propensities in community j are recomputed.

The synchronization mechanism couples these parts in continuous time. Micro engines advance their local CTMCs and export updated summaries $(\bar{i}_i(t), \bar{s}_i(t))$. The macro layer accumulates the total hazard $\Theta(t)$ from these summaries. If the macro hazard budget is reached, the simulator interpolates the macro event time, selects the source–destination pair, applies the import placement rule, and updates the affected micro engine. This handshake is the mechanism by which the two-scale process replaces explicit cross-community CTMC channels by macro-generated imports.

2.3 Micro Layer: Intra-Community CTMC Dynamics

The micro simulator is the local stochastic engine of the two-scale framework. Its role is to represent intra-community CTMC dynamics explicitly on the contact graph of a single community, while the interaction between communities is handled separately at the macro layer. In the present work, the micro layer follows the logic of the classical continuous-time Gillespie stochastic simulation algorithm [Gillespie, 1977], and is based on the efficient network-oriented implementation described in detail in [Kuryliak et al., 2025].

Let community i be represented by the local graph $G_i = (V_i, E_{ii})$, where nodes are individuals and edges are potential transmission contacts inside the community. At time t, each node is in one of the states S, I, R. The local state may therefore be written either as the node-level configuration $X_i(t)$ or through the sets $S_i(t)$, $I_i(t)$, and $R_i(t)$ defined above. The micro layer evolves through elementary stochastic events. For the internal SIR dynamics considered here, these are infection and recovery; other compartmental transitions can be added with analogous propensities:

$$S + I \xrightarrow{\beta_{\text{in}}} I + I, \qquad I \xrightarrow{\gamma} R,$$

where β_{in} is the within-community infection rate and γ is the recovery rate.

The Gillespie construction is based on event rates. For each susceptible node $v \in S_i(t)$ and each infectious node $u \in I_i(t)$, transmission along an active oriented contact $u \to v$ occurs at rate β_{in}. Equivalently, the total infection rate of susceptible node v is

$$\lambda_v^{\text{inf}}(t) = \beta_{\text{in}}\, m_v(t), \quad \lambda_u^{\text{rec}}(t) = \gamma.$$

where $m_v(t)$ is the number of infectious neighbors of v at time t. Thus, the infection rate of a node is determined by its current local infectious neighborhood. The coefficients β_{in} and γ are the within-community infection and recovery rates.

Summing over nodes gives the type-specific local rates

$$\theta_i^{\inf}(t) = \sum_{v \in S_i(t)} \lambda_v^{\inf}(t), \qquad \theta_i^{\mathrm{rec}}(t) = \sum_{u \in I_i(t)} \lambda_u^{\mathrm{rec}}(t) = \gamma i_i(t),$$

Since all infectious nodes recover with the same rate γ, this recovery selection is uniform over the currently infectious nodes.

Hence the total local event rate

$$\theta_i(t) = \theta_i^{\inf}(t) + \theta_i^{\mathrm{rec}}(t).$$

Here $\theta_i(t)$ is the total event rate of community i at the micro layer.

Given the current local state, if $\theta_i(t) > 0$, the waiting time to the next internal event is sampled from the exponential law

$$\Delta t_i = \frac{-\ln U}{\theta_i(t)}, \qquad U \sim \mathcal{U}(0,1).$$

If $\theta_i(t) = 0$, then no internal event is scheduled until a later external import or parameter change makes the local rate positive. A second uniform random variable $U_2 \sim \mathcal{U}(0,1)$ is then used to choose the event type according to the relative propensities:

$$\text{infection if } U_2\,\theta_i(t) < \theta_i^{\inf}(t), \qquad \text{recovery otherwise.}$$

Once the event type has been selected, a specific node e is chosen at random with probability proportional to its corresponding rate $\lambda_e^{\mathrm{event}}(t)$. Thus, for an event of the given type,

$$\mathbb{P}(\text{event on node } e \text{ next} \mid \text{event}) = \frac{\lambda_e^{\mathrm{event}}(t)}{\theta_i^{\mathrm{event}}(t)}.$$

After the event has been chosen, the local state is updated and only the affected node rates need to be refreshed. This event-driven update is the main advantage of the efficient Gillespie implementation used in [Kuryliak et al., 2025]: rather than rebuilding the full set of propensities after each event, the simulator updates only those quantities changed by the local state transition. In this way, the micro layer preserves exact stochastic intra-community dynamics while remaining suitable as the local building block of the larger two-scale simulator.

2.4 Macro Layer: Inter-Community Transmission

At the macro layer, the system is represented as a stochastic process on the graph of communities. Each community is treated as a meta-node, and the microscopic inter-community contacts of the original network are replaced by weighted macro-links that summarize the effective coupling between communities. In this way, the macro graph does not attempt to resolve individual cross-community transmissions one by one. Instead, it describes inter-community transmission through aggregate stochastic events between meta-nodes, while the detailed epidemic evolution inside each community remains delegated to the micro layer.

For every community i, the micro layer exports the normalized state descriptors

$$\bar{i}_i(t) = \frac{i_i(t)}{N_i}, \qquad \bar{s}_i(t) = \frac{s_i(t)}{N_i}.$$

Here $\bar{i}_i(t)$ denotes the current contagiousness of community i, measured through its infectious fraction, and $\bar{s}_i(t)$ denotes its current susceptibility, measured through its susceptible

fraction. If W_{ij} is the weight of the macro-link from community i to community j, then the instantaneous rate at which community i seeds an infection in community j is modeled as

$$\lambda_{ij}(t) = \beta_{\text{out}}\, W_{ij}\, \bar{i}_i(t)\, \bar{s}_j(t),$$

where β_{out} is the macro-level infection-rate parameter. If needed, this expression may be supplemented by a calibration factor to align macro-layer transmission with the corresponding micro-layer scale, but the central structure of the model is already captured by the formula above.

The total macro intensity is

$$\Theta(t) = \sum_{i \neq j} \lambda_{ij}(t).$$

When a macro event fires, the ordered pair $i \to j$ is selected with probability $\lambda_{ij}(t)/\Theta(t)$. The destination node is then selected from $S_j(t)$ according to the import placement distribution $\Pi_{ij}(\cdot \mid t)$, and the selected susceptible node is changed to state I. This placement rule is part of the simulator definition, because different choices of Π_{ij} can lead to different subsequent local CTMC paths.

Modeling assumption. The macro rate $\lambda_{ij}(t)$ is treated as an effective inter-community transmission intensity. It is not assumed to be an exact coarse-graining of the full microscopic inter-community edge set unless this is stated explicitly. In particular, the formula assumes that the retained macro variables $\bar{i}_i(t)$, $\bar{s}_j(t)$, and W_{ij} contain enough information to approximate the aggregate transmission pressure from community i to community j. When inter-community transmission depends on the states of specific boundary or bridge nodes, this assumption may fail and becomes a source of approximation error.

It is important to interpret this rate as an effective macro-level approximation rather than, in general, an exact coarse-graining of the full microscopic network. The exact inter-community intensity depends on which specific boundary nodes are infectious and which cross-community edges are active. By contrast, the macro rate retains only aggregate community-level quantities. This distinction is central for the accuracy analysis in Section 3.

A macro event has a simple interpretation: one imported infection is seeded in the destination community j. Thus, the macro layer does not specify which exact individual transmitted infection across communities; rather, it represents the cumulative effect of inter-community coupling through community-to-community stochastic jumps. A crucial point is that the macro rates are not static. Even if no macro event occurs, the micro-layer states within communities continue to evolve in continuous time. Since the micro layer is a CTMC, the summaries $\bar{i}_i(t)$ and $\bar{s}_j(t)$, and hence the rates $\lambda_{ij}(t)$, are typically piecewise constant between micro events and jump when a local event or import changes a community state. Consequently, the macro-event process should not be described as an ordinary NHPP with deterministic intensity. Conditional on the micro-layer trajectories, and therefore on the predictable path of $\Theta(t)$, macro events are generated by the cumulative-hazard construction. Unconditionally, the macro layer is a point process with stochastic predictable intensity $\Theta(t)$. Because $\Theta(t)$ varies along the sample path, the waiting-time construction must be generalized beyond the classical fixed-rate Gillespie draw, which motivates the next subsection.

2.5 Conditional NHPP Construction and Relation to Classical Gillespie

The classical Gillespie algorithm generates the waiting time to the next event exactly when the total event rate is constant. If the total rate is θ, then the waiting time is exponentially distributed with parameter θ, and may be sampled as

$$\Delta t = \frac{-\ln U}{\theta}, \qquad U \sim \mathcal{U}(0,1).$$

This formula can also be interpreted through accumulated hazard. In the constant-rate case, the accumulated hazard grows linearly with time, so the event time is obtained by waiting until the linear hazard reaches a random threshold. In this sense, the usual Gillespie draw already has an integral interpretation, even though for constant rates the integral reduces to a simple division.

At the macro layer considered here, however, the direct fixed-rate draw is not sufficient over an interval containing micro events. The quantities $\bar{i}_i(t)$ and $\bar{s}_j(t)$ evolve along the micro-layer CTMC path, so the inter-community rates $\lambda_{ij}(t)$, and therefore

$$\Theta(t) = \sum_{i \neq j} \lambda_{ij}(t),$$

are not constant. Conditional on the realized micro-layer trajectories, and therefore on the predictable path of $\Theta(t)$, macro events are generated by the usual cumulative-hazard construction. Unconditionally, the macro layer is not an ordinary NHPP with deterministic intensity; it is a point process with stochastic predictable intensity $\Theta(t)$. Thus, conditionally on the micro-layer paths, we measure elapsed risk through the **cumulative hazard**

$$H(t) = \int_{t_0}^{t} \Theta(s)\, ds,$$

where t_0 is the time of the previous macro event. We then draw one uniform random variable $U \sim \mathcal{U}(0,1)$ and define

$$E = -\ln U,$$

which we interpret as a random **hazard budget**. If $\Theta(t) = 0$ over part of the trajectory, the cumulative hazard is flat on that part; no macro event can occur until positive hazard has accumulated.

The next macro event occurs at the first time $t^* > t_0$ such that

$$H(t^*) = E.$$

Intuitively, the process keeps accumulating hazard over time, and the event fires exactly when this accumulated hazard reaches its random budget. This is the inverse, or integral, form of the usual Gillespie time-generation rule, generalized to dynamically varying rates. The classical formula is recovered as a sanity check: if $\Theta(t) \equiv \theta$, then

$$H(t) = \theta \cdot (t - t_0),$$

and solving $H(t^*) = E$ gives

$$t^* - t_0 = \frac{E}{\theta} = \frac{-\ln U}{\theta},$$

which is exactly the standard Gillespie waiting-time expression. This formulation therefore preserves the logic of Gillespie's SSA while extending it to the stochastic, microstate-dependent macro rates required by the two-scale model.

2.6 Synchronization Between Layers

The two layers of the simulator are coupled through a bidirectional handshake. Synchronization is necessary because the macro layer does not possess an autonomous state of its own: its transmission intensities depend on the evolving epidemic state inside the communities, while the micro layer must react whenever the macro layer generates an imported infection. The purpose of synchronization is therefore to keep the aggregate inter-community process and the local intra-community CTMCs consistent in time.

In the forward direction, each community is first advanced at the micro layer over a synchronization interval dt (or, equivalently, over a micro interval τ_{micro}). At the end of that interval, the community exports refreshed summary quantities $\bar{i}_i(t)$ and $\bar{s}_i(t)$.

Thus, after each micro update, the macro layer receives a new snapshot of the current inter-community transmission pressure and continues hazard accumulation using the updated total rate. In the present formulation there is one current macro hazard budget E, associated with the total hazard $\Theta(t)$, rather than separate budgets for individual macro-links.

If the accumulated macro hazard does not reach the budget during the current synchronization interval, the simulation simply advances to the next synchronization point, where the micro states are refreshed again and the procedure repeats. If, however, the cumulative hazard reaches the budget inside the current interval, then the macro event occurs at an interpolated time t^* within that interval. At that moment, one inter-community transmission is realized, a destination community is selected according to the current macro intensities, and one imported infection is inserted into the corresponding micro model. Conceptually, this means that the current interval is split: the destination community evolves up to t^*, receives the external infection at t^*, and then continues from the updated state over the remainder of the interval.

This backward coupling requires a consistent update of the local micro-layer clock. Suppose that, before the import, the destination community was evolving from local time t_0 with total internal rate $\theta_i(t_0)$ and a local random budget

$$E_i = -\ln U_i, \qquad U_i \sim \mathcal{U}(0,1).$$

If $\theta_i(t_0) > 0$, then under the usual Gillespie logic the next internal event would have been scheduled at

$$t_i^{\text{old}} = t_0 + \frac{E_i}{\theta_i(t_0)},$$

because the local state, and hence the rate, remain constant between two consecutive local events. If $\theta_i(t_0) = 0$, no internal event is scheduled before an external import changes the state. If $\theta_i(t_0) > 0$ and an imported infection arrives earlier, at time $t^* < t_i^{\text{old}}$, then the previously scheduled local event is interrupted. Up to time t^*, the local process has already consumed the hazard

$$H_i(t^*) = \int_{t_0}^{t^*} \theta_i(s)\, ds.$$

Since $\theta_i(s) = \theta_i(t_0)$ on $[t_0, t^*)$ in the standard Gillespie setting, this becomes

$$H_i(t^*) = \theta_i(t_0)\,(t^* - t_0).$$

The remaining local budget after the import is therefore

$$E_i^{\text{res}} = E_i - \theta_i(t_0)\,(t^* - t_0).$$

At time t^*, the imported infection changes the local state instantaneously, so the old rate is no longer valid. Let $\theta_i^{\text{new}}(t^*)$ denote the updated total local rate after the import has been applied and the local propensities have been recomputed. If $\theta_i^{\text{new}}(t^*) > 0$, the new next-event time, denoted by t_i^{new}, is obtained by continuing from the remaining budget under the updated rate:

$$t_i^{\text{new}} = t^* + \frac{E_i^{\text{res}}}{\theta_i^{\text{new}}(t^*)}.$$

If $\theta_i^{\text{new}}(t^*) = 0$, the local clock is instead inactive until a later import or state change creates a positive rate. Thus, in a random-time-change implementation, t_i^{new} can be computed by subtracting the already consumed hazard from the original budget and then propagating the local clock forward under the updated state. Because exponential clocks are memoryless, this residual-budget update is distributionally equivalent to resampling a fresh exponential budget after the import; the residual form is useful because it makes the consumed hazard explicit. If another external import arrives before t_i^{new}, the same residual-budget update is applied again. In this way, the synchronization rule is not only an exchange of summary statistics, but also a time-consistency condition: the macro layer advances by accumulated hazard built from the evolving micro summaries, while each imported macro event instantaneously modifies the destination CTMC and forces a consistent recalculation of its local next-event time.

3 Accuracy and Error Decomposition

3.1 Accuracy Framework and Evaluation Logic

The key point of the accuracy analysis is that the two-scale approximation does not primarily modify the local stochastic dynamics inside communities. Instead, it modifies the interface between communities. Thus, the central question is not whether the local Gillespie engines are exact, but whether the macro layer contains enough information to reproduce the timing and placement of inter-community events generated by the full-network CTMC.

The purpose of this part of the paper is to assess when the two-scale simulator provides an accurate representation of the underlying continuous-time stochastic dynamics on the original graph, and to identify the mechanisms that govern its approximation error. Throughout this analysis, the full-network CTMC simulated by the Gillespie algorithm is treated as the reference process. The proposed simulator is therefore evaluated as an approximation to this microscopic reference: it preserves explicit intra-community dynamics at the micro layer, while replacing the full set of inter-community infection channels by a macro-level abstraction driven by community-level state summaries.

In this paper, *accuracy* is understood relative to the reference CTMC over a fixed time horizon. The comparison focuses on stochastic observables that are relevant for community-structured dynamics. These include inter-community transmission times, the ordering of arrivals across communities, prevalence trajectories within communities and at the network level, and the variability of these quantities across sample paths. Thus, accuracy is not reduced to agreement of mean trajectories alone. A useful abstraction should also reproduce, within the resolution of its macro state, the uncertainty and event-level stochastic patterns generated by the reference process.

This evaluation logic is motivated by the role of rare and heterogeneous inter-community transmissions in community-structured graphs. Two simulators may produce similar mean prevalence curves while differing in the timing or conditional structure of inter-community infections. Such differences can change the subsequent local trajectories

inside affected communities and may therefore alter the distribution of later observables. For this reason, the present work does not restrict attention to deterministic limits or mean-field summaries. Instead, it asks whether the two-scale abstraction captures the relevant stochastic behavior of the full CTMC at the level of event times, community-level paths, and derived quantities such as arrival times and outbreak sizes.

The remainder of this part is organized around benchmark cases designed to expose specific aspects of the approximation. We begin with analytically tractable graph motifs where the inter-community mechanism can be inspected directly at the edge level. We then move to structured community-based graph families in which the interaction between intra-community dynamics and inter-community transmission is shaped by graph topology. These benchmarks provide a controlled setting in which discrepancies between the two-scale simulator and the full-network reference can be attributed to identifiable structural features.

Standing assumptions for the accuracy analysis. Unless stated otherwise, the full-network process is a continuous-time Markov chain on a fixed finite graph $G = (V, E)$, with node states in $\{S, I, R\}$ or in $\{S, I\}$ for the SI benchmark cases. The node set is partitioned into fixed communities

$$V = \bigsqcup_{i=1}^{M} V_i.$$

Within-community transitions are generated by the usual Gillespie/SSA propensities on the induced subgraphs $G_i = G[V_i]$. Inter-community edges are represented in the full reference process by the sets E_{ij}, while the two-scale simulator replaces these edge-level channels by macro-level import events. All stochastic clocks are exponential conditional on the current state, and all rates are assumed constant between consecutive CTMC events unless a time-dependent macro hazard is explicitly specified. The full-network Gillespie CTMC is used as the reference process for all accuracy statements.

3.2 Sources of Approximation Error

The two-scale simulator is designed to preserve the local CTMC dynamics within each community while replacing the original inter-community mechanism by a macro-level abstraction and a coupling rule. At the modeling level, it departs from the full-network Gillespie process through the way inter-community transmissions are represented and injected into the micro layer. At the implementation level, additional numerical error may arise from synchronization and hazard evaluation.

Let

$$V = \bigsqcup_{i=1}^{M} V_i$$

be the fixed partition of the original graph into communities. Let $G_i = G[V_i]$ be the subgraph induced by community i, and let

$$E_{ij} = \{(u, v) \in E : \ u \in V_i, \ v \in V_j\}$$

denote the set of inter-community edges from community i to community j. Denote by $X(t)$ the full-network CTMC state and by $X_i(t)$ its restriction to V_i. In the full-network process, communities are coupled through edge-level inter-community events in the sets E_{ij}. In the two-scale simulator, the same coupling is represented by a macro-generated import process into the local community models.

Proposition (Conditional micro-layer exactness). Fix the initial microstate in each community. Also fix the complete import process into every community, including all import times and all susceptible target nodes in the destination communities. Conditional on this import process, the micro layer in the two-scale simulator evolves each $X_i(t)$ exactly as the Gillespie CTMC on G_i, with the same intra-community transitions and with the specified imports applied as exogenous $S \to I$ jumps. Consequently, any approximation error relative to the full-network Gillespie reference arises from how the import process is generated and coupled, not from the local CTMC simulation itself.

Proof. Once the import times and target nodes are fixed, the evolution inside each community is determined by the same local transition rules as in the full-network CTMC restricted to G_i. Between two exogenous imports, the micro layer applies the Gillespie algorithm to the same intra-community reaction channels and propensities. At an import time, the prescribed target node is updated by the same exogenous state transition. Therefore the sample-path law of $X_i(t)$, conditional on the fixed import process, coincides with the corresponding local CTMC law. The remaining discrepancy can only come from the construction of the import process itself, including its rate, target placement, and any numerical treatment of its timing.

Interpretation. This proposition formalizes the main structural message of the paper: within-community exactness does not imply exactness of the full two-scale process. The local Gillespie engines are exact conditional on the imports they receive, but the imports themselves are generated by a coarse-grained macro mechanism. Therefore, the dominant approximation error is an interface error between the micro and macro layers. In what follows, we decompose this discrepancy into three sources: the rate abstraction that replaces the microscopic inter-community intensity, the import placement rule that replaces the edge-conditioned target choice, and the numerical timing error introduced by synchronization and hazard evaluation.

3.2.1 Rate Abstraction Error

In the full-network CTMC, the instantaneous inter-community infection intensity from community i to community j is determined by the active cross-community infection opportunities in the original graph. For a full microscopic state x, the exact intensity is

$$a_{ij}^{\star}(x) = \beta_{\text{out}} \sum_{(u,v)\in E_{ij}} w_{uv}\, \mathbf{1}\{x_u = I,\ x_v = S\}. \tag{1}$$

The two-scale simulator replaces this edge-level quantity by the macro-rate

$$\lambda_{ij}(x) = \beta_{\text{out}}\, W_{ij}\, \bar{i}_i(x)\, \bar{s}_j(x), \qquad \bar{i}_i(x) = \frac{i_i(x)}{N_i}, \quad \bar{s}_j(x) = \frac{s_j(x)}{N_j}, \tag{2}$$

where W_{ij} is an aggregate weight associated with the meta-edge (i, j). The macro state therefore retains only community-level fractions, not the exact boundary configuration of the original graph.

Proposition (Rate abstraction). Define the rate abstraction error process by

$$\varepsilon_{ij}(t) = a_{ij}^{\star}(X(t)) - \lambda_{ij}(X(t)).$$

If the macro-rate $\lambda_{ij}(x)$ is exact for all full-network states in a given state space, then the microscopic cross-community intensity $a^\star_{ij}(x)$ must be completely determined by the retained aggregates $(\bar{i}_i(x), \bar{s}_j(x), W_{ij})$. Equivalently, if there exist two states x and y with

$$\bar{i}_i(x) = \bar{i}_i(y), \qquad \bar{s}_j(x) = \bar{s}_j(y),$$

and the same fixed W_{ij}, but with different boundary or bridge configurations such that

$$a^\star_{ij}(x) \neq a^\star_{ij}(y),$$

then no macro-rate of the form (2) can be exact for both states.

Proof. The macro-rate $\lambda_{ij}(x)$ is a function only of $\bar{i}_i(x)$, $\bar{s}_j(x)$, W_{ij}, and the parameter β_{out}. Therefore, two full states with identical values of these quantities necessarily receive the same macro-rate. If their microscopic intensities $a^\star_{ij}$ differ, then the same value of λ_{ij} cannot equal both microscopic intensities. Hence exactness of the macro-rate over the relevant state space requires the true inter-community intensity to be determined by the retained macro variables.

Interpretation. Equation (1) depends on which specific nodes in V_i and V_j are incident to inter-community edges and on their current states. The macro summary $(\bar{i}_i, \bar{s}_j)$ discards this bridge-level structure. Thus, the approximation is reliable only when the boundary nodes are sufficiently representative of the communities with respect to inter-community exposure, or when the graph structure makes the aggregate summaries informative enough for the true inter-community intensity. When inter-community edges are concentrated on a small subset of boundary nodes, or when infection status is strongly correlated with boundary position, the error process $\varepsilon_{ij}(t)$ can be substantial.

3.2.2 Import Placement Error

In the full-network Gillespie process, an inter-community event $i \to j$ occurs through a specific active edge. Conditional on observing such an event at state x, the target node in V_j is not chosen only from the community-level susceptible fraction. It is determined by the firing edge $(u, v) \in E_{ij}$. Provided $a^\star_{ij}(x) > 0$, the full CTMC induces the edge-conditioned target distribution

$$\pi^\star_{ij}(v \mid x) = \frac{\mathbf{1}\{x_v = S\} \sum_{u \in V_i:\ (u,v) \in E_{ij}} \beta_{uv}\, \mathbf{1}\{x_u = I\}}{a^\star_{ij}(x)}, \qquad v \in V_j. \tag{3}$$

Here $\beta_{uv} = \beta_{\text{out}} w_{uv}$ under the weighted-edge parametrization in (1). Consequently, the induced distribution over target nodes $v \in V_j$ depends on the inter-community adjacency of v and on the states of its cross-community neighbors.

The two-scale simulator instead injects an imported infection into V_j by an abstracted community-level seeding rule. Let $\Pi_{ij}(t)$ denote the simulator's conditional distribution over susceptible target nodes in V_j, given that a macro event $i \to j$ fires at time t. A simple choice is uniform seeding over susceptible nodes in V_j, but the following criterion applies to any abstracted seeding distribution.

Proposition (Import placement). Fix a state x and condition on the event that an inter-community transmission $i \to j$ occurs at that state. The import placement mechanism in the two-scale simulator reproduces the immediate post-event microstate

law of the full-network CTMC if and only if the seeding distribution $\Pi_{ij}(\cdot \mid x)$ coincides with the full-network conditional distribution $\pi^{\star}_{ij}(\cdot \mid x)$ in (3). If these distributions differ, then the two-scale simulator changes the distribution of the destination microstate immediately after import.

Proof. Conditioning on the occurrence of an $i \to j$ event fixes the event type at the community level but not the target node in V_j. In the full-network CTMC, the post-event state in V_j is obtained by infecting the node selected by the edge-level firing distribution $\pi^{\star}_{ij}(\cdot \mid x)$. In the two-scale simulator, it is obtained by infecting a node sampled from $\Pi_{ij}(\cdot \mid x)$. The two post-event laws coincide exactly when these two target-node distributions coincide. Otherwise, at least one susceptible node receives different probability under the two mechanisms, and the law of the post-event microstate is perturbed.

Interpretation. Import placement error is a coupling error, not a minor implementation detail. Even if the macro event time were correct, seeding the infection into a uniformly random susceptible node rather than into the edge-conditioned target can modify the early intra-community dynamics after import. This effect is expected to be most visible in motifs where inter-community connectivity is concentrated on a small set of boundary nodes, because the full-network process necessarily targets that subset whereas an abstracted rule may distribute imports more broadly.

3.2.3 Synchronization and Numerical Approximation Error

The structural errors above arise from the two-scale abstraction itself. A separate, secondary source of error may arise from the numerical realization of synchronization. The macro layer generates inter-community events from the total rate

$$\Theta(t) = \sum_{i \neq j} \lambda_{ij}(t).$$

In the ideal continuous-time formulation, if $\Theta(t)$ were known exactly and the macro event time $t^{\star}$ were obtained from

$$\int_{t_0}^{t^{\star}} \Theta(s)\, ds = E, \qquad E \sim \mathrm{Exp}(1),$$

then the event time would be exact conditional on the realized micro-layer trajectories that determine $\Theta(t)$. Unconditionally, this is a macro point process with stochastic predictable intensity $\Theta(t)$, not a deterministic-intensity NHPP. In practice, however, $\Theta(t)$ is updated through synchronization between layers, and the cumulative hazard is accumulated numerically.

Proposition (Synchronization timing error bound). Let

$$H(t) = \int_{t_0}^{t} \Theta(s)\, ds$$

be the exact cumulative hazard, and let $\widehat{H}(t)$ be the numerically accumulated hazard used by the implementation. Let $t^{\star}$ and $\widehat{t}$ be defined by

$$H(t^{\star}) = E, \qquad \widehat{H}(\widehat{t}) = E.$$

Assume that, on an interval containing $t^\star$ and $\widehat{t}$,

$$|H(t) - \widehat{H}(t)| \le \eta$$

and

$$\Theta(t) \ge \theta_{\min} > 0.$$

Then the event-time error is bounded by

$$|\widehat{t} - t^\star| \le \frac{\eta}{\theta_{\min}}.$$

Proof. Since H is increasing with derivative $\Theta(t) \ge \theta_{\min}$, we have

$$|H(\widehat{t}) - H(t^\star)| \ge \theta_{\min}|\widehat{t} - t^\star|.$$

On the other hand, $H(t^\star) = E = \widehat{H}(\widehat{t})$, so

$$|H(\widehat{t}) - H(t^\star)| = |H(\widehat{t}) - \widehat{H}(\widehat{t})| \le \eta.$$

Combining the two inequalities gives the bound.

Interpretation. The resulting timing discrepancy is controlled by the error in approximating the cumulative hazard H, including quadrature over synchronization intervals, interpolation of the threshold crossing time, and any delay in updating $\bar{i}_i(t)$ and $\bar{s}_j(t)$ relative to the underlying micro-layer evolution. Synchronization and numerical timing error is not the primary approximation mechanism of the two-scale simulator. It can, however, add timing error on top of the structural discrepancies described above. For this reason, the benchmark analysis focuses primarily on rate abstraction and import placement, while the numerical implementation of hazard accumulation must remain consistent with the intended stochastic-intensity macro point process.

3.3 Clique–bridge–leaf benchmark

The clique–bridge–leaf benchmark is used as a minimal counterexample to the idea that aggregate community fractions are always sufficient for inter-community transmission. Although the source community is internally well mixed as a clique, the export event depends on the infection status of a small bridge set. This makes the motif analytically simple while still exposing the central interface error of the two-scale approximation.

This benchmark is introduced as a minimal test for identifying exactly what is lost when passing from the full microscopic network to the two-scale abstraction. In the previous sections, the micro layer preserves the exact CTMC/Gillespie dynamics inside each community, while the macro layer describes inter-community transmission through the aggregated intensity

$$\lambda_{ij}(t) = \beta_{\text{out}} W_{ij} \bar{i}_i(t) \bar{s}_j(t).$$

It is precisely this difference between the exact states of individual nodes and the aggregated community-level variables that is central for the present benchmark.

3.3.1 Graph construction and motivation

Consider two communities. Community 0 is the infection source and has local graph

$$G_0 = (V_0, E_0).$$

In this benchmark, it is a clique

$$K_n,$$

so that

$$N_0 = n$$

nodes belong to the source community. Community 1 is the receiver and consists of a single leaf node A. This node has no internal dynamics before the import event; it can only become infected through an inter-community contact from community 0.

For analytical clarity, this benchmark uses the irreversible SI version of the local process, so that once a node becomes infected, it remains infectious. Unless explicitly conditioned otherwise, the analytical bridge-finding formulas below assume that the initial infected node in the clique is chosen uniformly from V_0; if the initial seed lies in B, then $T_B = 0$.

Inside the clique K_n, we distinguish a bridge set of nodes

$$B \subset V_0, \qquad |B| = k.$$

The leaf A is connected only to the nodes in B. These bridge nodes are therefore the only microscopic nodes of community 0 through which infection can reach A. In the full microscopic network, transmission from community 0 to community 1 is possible not from any infected node of the clique, but only from infected bridge nodes.

Let

$$B_I(t) = \sum_{b \in B} \mathbf{1}\{X_b(t) = I\}$$

be the number of infected bridge nodes at time t. This is the key microscopic quantity for the export event: the leaf is infected by the concrete states of the bridge nodes, not by the average infected fraction in the clique.

3.3.2 Exact Micro dynamics

Proposition. Exact bridge-dependent micro hazard. In the full Micro model, the instantaneous import intensity into the leaf node A is

$$a_{01}^{\mathrm{Micro}}(t) = \beta_{\mathrm{out}} B_I(t),$$

provided that node A is still susceptible. If A is already infected, further import into it is not considered.

This statement follows directly from the local graph structure. The leaf node A has contacts only with bridge nodes. Each infected bridge node creates one active infectious edge to A with rate β_{out}. If there are $B_I(t)$ infected bridge nodes, then the total intensity of these parallel edges is $\beta_{\mathrm{out}} B_I(t)$.

This formula has an important interpretation. Even if many nodes in the clique are already infected, if no bridge node has yet been infected, then

$$B_I(t) = 0$$

and therefore

$$a_{01}^{\mathrm{Micro}}(t) = 0.$$

Thus, in the exact microscopic model, the leaf node is fully protected until the infection actually reaches at least one bridge node.

Let the time of first bridge activation be denoted by

$$T_B = \inf\{t : B_I(t) > 0\}.$$

In this benchmark, it is also convenient to interpret it as the bridge-finding time:

$$T_{\text{find}} = T_B.$$

3.3.3 Two-phase structure of the exact process

Proposition. Two-phase structure for a single bridge and a single leaf. Assume $k = 1$, SI dynamics, and no recovery. Let T_B be the infection time of the bridge node and let T_A be the infection time of the leaf node. Then the exact Micro process has the decomposition

$$T_A = T_B + Z, \qquad Z \sim \text{Exp}(\beta_{\text{out}}),$$

where Z is the post-bridge waiting time, conditionally on the history up to T_B. Consequently,

$$\mathbb{E}[T_A] = \mathbb{E}[T_B] + \frac{1}{\beta_{\text{out}}}.$$

The first phase is the delay phase. The infection spreads inside the clique, but the leaf node still has no active infectious contact. Over the whole interval

$$0 \le t < T_B,$$

the risk to the leaf node is zero.

The second phase begins after the unique bridge node enters state I. From that moment, the leaf node sees exactly one active bridge–leaf edge with intensity β_{out}. Since in the SI model the infected bridge node does not recover, the subsequent dynamics inside the clique no longer change the hazard for this one leaf node. Therefore, after the bridge node becomes infected, the additional waiting time is exponential with mean

$$\frac{1}{\beta_{\text{out}}}.$$

This decomposition highlights an important property of the exact Micro model: the size of the clique and its internal dynamics influence the infection time of the leaf node only through the delay phase, that is, through the time needed to infect the bridge node. Once the unique bridge node is already infected, the remaining events inside the clique no longer affect the expected infection time of this one leaf node; it is determined only by a single active edge with rate β_{out}.

For several bridge nodes, this interpretation remains useful, but the second phase may already change. If the leaf node has contacts with several bridge nodes, then after additional bridge nodes become infected, the intensity becomes

$$a_{01}^{\text{Micro}}(t) = \beta_{\text{out}} B_I(t).$$

That is, several infected bridge nodes create several parallel active edges to the leaf node. In that case, the post-bridge time is no longer necessarily equal to $1/\beta_{\text{out}}$, because the later infection of additional bridge nodes may increase the hazard and shorten the second phase.

Let the import time into the leaf node A be denoted by

$$T_A.$$

Then the exact infection clock of the leaf node is given through the accumulated hazard:

$$\int_0^{T_A} \beta_{\text{out}} B_I(s)\, ds = -\ln U, \qquad U \sim \mathcal{U}(0,1).$$

Up to time T_B, this integral does not accumulate, because $B_I(s) = 0$. After the first bridge node becomes infected, it starts growing. If further bridge nodes become infected later, the accumulation speed increases, since $B_I(t)$ becomes larger.

3.3.4 Bridge-finding time in a clique

Proposition. Expected bridge-finding time in a clique. For the case of one bridge node, a simple analytical check can be obtained. Under the uniform initial seeding convention stated above, if $k = 1$, then the expected time until the bridge node becomes infected is

$$\mathbb{E}[T_B] = \frac{H_{n-1}}{\beta_{\text{in}} n},$$

where

$$H_{n-1} = \sum_{m=1}^{n-1} \frac{1}{m}$$

is the harmonic number.

After the bridge node becomes infected, the leaf node receives one active infectious contact with rate β_{out}, so the additional expected time until import is

$$\frac{1}{\beta_{\text{out}}}.$$

Hence,

$$\mathbb{E}[T_A^{\text{Micro}}] = \frac{H_{n-1}}{\beta_{\text{in}} n} + \frac{1}{\beta_{\text{out}}}.$$

For an arbitrary number of bridge nodes k, under the same uniform initial seeding convention, it is convenient to consider the time of first activation of the bridge set. Its expected value can be written as a sum over clique states:

$$\mathbb{E}[T_B] = \sum_{m=1}^{n-k} \frac{\binom{n-k}{m}}{\binom{n}{m}} \cdot \frac{1}{\beta_{\text{in}} m(n-m)}.$$

Here the factor

$$\frac{\binom{n-k}{m}}{\binom{n}{m}}$$

is the probability, under exchangeability of the infected set in the clique, that among the first m infected nodes there is still no bridge node. If k increases, this probability decreases rapidly, so the first bridge node is activated earlier. In the limiting case of large k, the exact hazard becomes closer to the macro hazard, because the bridge nodes better represent the whole community. If one instead conditions the initial seed to be outside B, the same calculation uses the corresponding conditional no-bridge probabilities rather than the unconditional factor shown here.

3.4 Mean-field versus exact hazard bias

This subsection explains why an apparently reasonable macro hazard may still distort event timing. The macro hazard can be correct as a conditional mean of the exact microscopic bridge hazard, but event times depend on accumulated hazards and survival probabilities, not only on instantaneous expected rates. Consequently, replacing the random bridge-dependent hazard by its mean-field counterpart can change the distribution of export times.

The clique–bridge–leaf benchmark separates the exact microscopic mechanism from its aggregate approximation. The exact model depends on the state of the bridge nodes, while the macro model sees only the community-level infected fraction $\bar{i}_0(t)$. This subsection makes this difference explicit and explains why replacing the exact hazard by a mean-field hazard changes the timing of export events.

3.4.1 Macro approximation of bridge risk

For comparison with the exact Micro process before the leaf has been infected, the macro layer does not see the specific bridge nodes. It receives only the aggregated infected fraction $\bar{i}_0(t)$. If we interpret the inter-community weight W_{01} as corresponding to the number of potential bridge contacts, that is,

$$W_{01} = k,$$

then the macro rate for import from community 0 to community 1 takes the form

$$\lambda_{01}^{\mathrm{Macro}}(t) = \beta_{\mathrm{out}} k \bar{i}_0(t) \bar{s}_1(t).$$

Since in the benchmark community 1 consists of a single susceptible leaf node before import, we may take

$$\bar{s}_1(t) = 1.$$

Therefore, the macro rate simplifies to

$$\lambda_{01}^{\mathrm{Macro}}(t) = \beta_{\mathrm{out}} k \frac{i_0(t)}{n}.$$

This is the aggregated version of the bridge risk. It replaces the exact number of infected bridge nodes $B_I(t)$ by its mean-field estimate:

$$B_I(t) \approx k \frac{i_0(t)}{n}.$$

Conditional on the aggregate path $\{i_0(s) : s \geq 0\}$, the corresponding macro clock is given by

$$\int_0^{T_A^{\mathrm{Macro}}} \beta_{\mathrm{out}} k \frac{i_0(s)}{n}\, ds = -\ln U, \qquad U \sim \mathcal{U}(0,1).$$

3.4.2 Conditional-mean interpretation

Proposition. Macro hazard is the conditional mean of the exact micro hazard. Under the exchangeability convention for the infected set in the clique, and before conditioning on the absence of a previous export event, the macro rate is the conditional mean of the exact micro hazard:

$$\mathbb{E}\left[a_{01}^{\mathrm{Micro}}(t) \mid i_0(t)\right] = \lambda_{01}^{\mathrm{Macro}}(t).$$

Indeed, if we only know the number of infected nodes in the clique $i_0(t)$, but not their identities, and the infected set is exchangeable among all subsets of size $i_0(t)$, then the probability that a particular bridge node is infected equals

$$\frac{i_0(t)}{n}.$$

Since there are k bridge nodes, the expected number of infected bridge nodes is

$$\mathbb{E}[B_I(t) \mid i_0(t)] = k\frac{i_0(t)}{n}.$$

Then

$$\mathbb{E}\left[a_{01}^{\text{Micro}}(t) \mid i_0(t)\right] = \mathbb{E}\left[\beta_{\text{out}} B_I(t) \mid i_0(t)\right] = \beta_{\text{out}} k\frac{i_0(t)}{n} = \lambda_{01}^{\text{Macro}}(t).$$

Thus, the macro rate is the correct conditional mean of the exact micro hazard. However, this does not mean that the macro model reproduces the exact infection-time distribution. The reason is that the import time is determined not by the hazard itself, but by the exponential of the accumulated hazard.

3.4.3 Bias in event timing

The early-export bias should be formulated at the level of accumulated hazards, not only at the level of instantaneous hazards. Let

$$X_t = \int_0^t \beta_{\text{out}} B_I(s)\, ds$$

be the exact accumulated bridge hazard up to time t. Let $\mathcal{F}_t^I$ denote the information retained by the macro layer up to time t, for example the aggregate infection-count path $\{i_0(s) : 0 \le s \le t\}$. Suppose that the macro accumulated hazard is the conditional mean

$$\bar{X}_t = \mathbb{E}\left[X_t \mid \mathcal{F}_t^I\right].$$

Under the exchangeability assumptions used above, this corresponds to

$$\bar{X}_t = \int_0^t \beta_{\text{out}} k\frac{i_0(s)}{n}\, ds.$$

Proposition. Conditional Jensen survival inequality. The exact conditional survival probability satisfies

$$\mathbb{E}\left[\exp(-X_t) \mid \mathcal{F}_t^I\right] \ge \exp(-\bar{X}_t).$$

Consequently, after taking expectations,

$$S^{\text{Micro}}(t) \ge S^{\text{Macro}}(t),$$

where

$$S^{\text{Micro}}(t) = \mathbb{E}[\exp(-X_t)], \qquad S^{\text{Macro}}(t) = \mathbb{E}\left[\exp(-\bar{X}_t)\right].$$

Proof. The function $x \mapsto \exp(-x)$ is convex. Applying Jensen's inequality conditionally on $\mathcal{F}_t^I$ gives

$$\mathbb{E}\left[\exp(-X_t) \mid \mathcal{F}_t^I\right] \ge \exp\left(-\mathbb{E}[X_t \mid \mathcal{F}_t^I]\right) = \exp(-\bar{X}_t).$$

Taking expectations gives the unconditional survival inequality.

Corollary. Expected import-time ordering. If the survival inequality holds for all $t \geq 0$ and the expectations are finite, then

$$\mathbb{E}[T_A^{\mathrm{Micro}}] \geq \mathbb{E}[T_A^{\mathrm{Macro}}].$$

Proof. Since the expectation of a nonnegative infection time can be represented as the area under the survival curve,

$$\mathbb{E}[T_A] = \int_0^\infty S(t)\, dt,$$

integrating the survival inequality over time gives the result.

Intuitively, this means that the macro model may create premature leakage of risk. As soon as infected nodes appear in the clique, the quantity $i_0(t)/n$ becomes positive, and the macro layer begins accumulating hazard for import. But in the full microscopic network, import may still be impossible if the infected nodes are not bridge nodes.

The clique–bridge–leaf benchmark is useful precisely because it isolates structural abstraction error from other sources of error. There is no complicated community structure here, no many-path interaction, and no heterogeneous internal topology. There is only a clique, k bridge nodes, and one leaf receiver. If even in such a simple system the macro approximation deviates from the full Gillespie reference, then the reason for the deviation is clear: the macro layer does not know the exact states of the bridge nodes.

3.5 Numerical benchmark on the clique–bridge–leaf graph

3.5.1 Experimental setup

In the implemented numerical benchmark, two models were compared: the full Micro simulation on the explicit network and the MicroMacro approximation. The parameterization corresponds to the case $n = 100$, $k = 2$, that is, the source community is a clique of 100 nodes, and inter-community transmission passes through two bridge nodes.

Since the source community is a clique, community 0 reaches near-complete infection very quickly in both models. This is expected, because the number of active $S-I$ edges is large during the initial and intermediate phases of the intra-community process. However, the main quantity of interest in this benchmark is not the visual shape of the infection curve inside the clique, but the timing of bridge activation and export. Therefore, the most informative comparison is given by the timing distributions in Figure 1.

3.5.2 Timing distributions

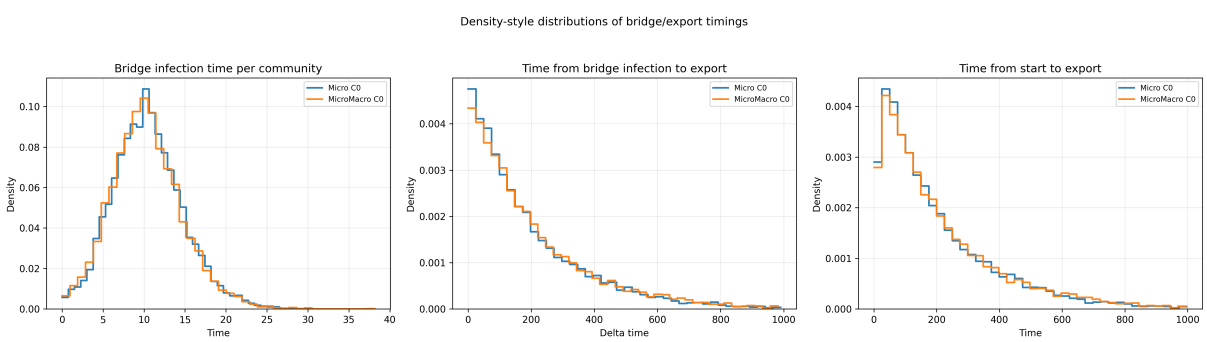


Figure 1: Density-style distributions of bridge and export timings.

Figure 1 separately shows the bridge infection time, the time from bridge infection to export, and the total time from simulation start to export. The first panel shows that the distributions of bridge infection times in Micro and MicroMacro are nearly identical. This is an important sanity check: both models reproduce a similar time scale for the activation of the bridge mechanism in the clique.

The second panel shows the distribution of the delay from bridge infection to export. It has a long right tail, corresponding to the exponential character of waiting for a rare inter-community transmission. The third panel shows the full time from the start to export; here too, the Micro and MicroMacro distributions are close.

3.5.3 Implications for the two-scale approximation

This result is important for the interpretation of the benchmark. Theoretically, the mean-field macro hazard may produce premature leakage of risk. However, in this specific configuration with $n = 100$ and $k = 2$, the effect is moderate. The reason is that the bridge nodes in the clique are activated rather early, while the main variability of the total export time is associated not with the bridge infection time, but with the long random delay between bridge activation and the export event itself. Thus, the difference between the exact and aggregate hazards is partly masked by the broader distribution of inter-community waiting times.

This does not negate the presence of structural bias. On the contrary, the benchmark shows in which regimes it may be small. If there are enough bridge nodes, if the clique mixes rapidly, and if the export rate is relatively small, then the macro approximation may agree well with the full Micro reference. In such a regime, the quantity

$$B_I(t)$$

quickly becomes close to its mean-field analogue

$$k\frac{i_0(t)}{n}.$$

Therefore, the macro hazard

$$\lambda_{01}^{\mathrm{Macro}}(t) = \beta_{\mathrm{out}} k \frac{i_0(t)}{n}$$

becomes an acceptable effective description of the exact micro hazard.

However, the same benchmark also indicates the direction for stress tests. To reveal premature leakage more strongly, one should consider cases with smaller k, slower internal spreading inside the community, or a higher inter-community transmission rate. In such regimes, the macro layer may begin accumulating risk substantially earlier than the full-network Gillespie process allows real export through the bridge nodes.

Thus, the clique–bridge–leaf benchmark serves two functions. First, it provides a simple analytical model for checking the consistency of the Micro and MicroMacro simulators. Second, it demonstrates that the macro rate based on $\bar{i}_i(t)$ is not an exact coarse-graining, but an effective approximation. Its accuracy depends on how well the bridge nodes represent the state of the whole community. For well-mixed communities and sufficiently broad bridge contacts, the approximation may be practically accurate. For narrow boundary bottlenecks, it may systematically accelerate inter-community transmission.

3.6 Chain–of–cliques benchmark

After the clique–bridge–leaf benchmark, the next natural test case is a chain of cliques. The previous benchmark analyzed a single local export event: the process had to leave one

clique through a bridge node and activate one leaf node. In two-scale modeling, however, it is also important to understand how such local timing errors behave when the process propagates through several communities in sequence.

We consider a chain of L communities,

$$G_0 \to G_1 \to \cdots \to G_{L-1}.$$

Each community $G_i = (V_i, E_i)$ is a clique of size

$$N_i = n.$$

The process starts in the first community G_0. A downstream community G_{i+1} becomes active only after a successful export event from G_i. Neighboring cliques are connected through a narrow interface, for example through k bridge nodes or through an equivalent set of sparse inter-community contacts.

This benchmark tests not an isolated local transition, but a propagation wave across a sequence of communities. If the timing error of one transition is small but repeated several times, it may accumulate along the chain. Therefore, the chain–of–cliques benchmark is a stronger test of the global timing error of the two-scale approximation.

3.6.1 Arrival times and propagation delays

Let the number of infected, or active, nodes in community i at time t be denoted by the already introduced count $i_i(t) = |I_i(t)|$. The first-arrival time in community i is defined as

$$T_i = \inf\{t : i_i(t) > 0\}.$$

For the initially active community we set

$$T_0 = 0.$$

The local propagation delay between two neighboring communities is

$$\Delta T_i = T_{i+1} - T_i, \qquad i = 0, \ldots, L-2.$$

The global first-arrival time in the last community is

$$T_{\text{last}} = T_{L-1}.$$

Proposition. Arrival time in the last community decomposes into local delays. On sample paths for which every community in the chain is eventually reached, the first-arrival time in the last community can be written as

$$T_{\text{last}} = \sum_{i=0}^{L-2} \Delta T_i.$$

If the expectations are finite, consequently

$$\mathbb{E}[T_{\text{last}}] = \sum_{i=0}^{L-2} \mathbb{E}[\Delta T_i].$$

This follows directly from the definition of the local propagation delays. Each ΔT_i measures the time needed to move from community G_i to community G_{i+1}. Summing all these delays gives the total time needed to reach G_{L-1}.

If all transitions are parametrized identically, that is, all cliques have the same size n, the same number of bridge nodes k, the same within-community rate β_{in}, the same inter-community rate β_{out}, and each newly activated community starts from the same one-node seeding distribution, then the local delays have the same distribution in the idealized first-passage baseline. This baseline treats each newly activated clique as evolving autonomously for purposes of its first export; if the full simulator allows repeated upstream imports into an already activated community, the identities below remain useful diagnostics but the one-transition formulas are no longer exact closed forms. In the homogeneous first-passage case,

$$\mathbb{E}[T_{\text{last}}] = (L-1)\mathbb{E}[\Delta T].$$

This identity is the main reason why the chain benchmark is useful: a local timing discrepancy of a single transition can scale with the number of transitions in the chain.

3.6.2 Expected time of one exact Micro transition

A single transition

$$G_i \to G_{i+1}$$

can be decomposed into two phases. The first phase is the time needed for the process inside G_i to reach at least one bridge node. We denote this time by

$$T_{\text{find}}.$$

The second phase is the time between the first bridge activation and the successful export event into the next community. We denote it by

$$T_{\text{cross}}.$$

Proposition. One exact Micro transition decomposes into a bridge-finding phase and a crossing phase. The local propagation delay satisfies

$$\Delta T = T_{\text{find}} + T_{\text{cross}},$$

and therefore

$$\mathbb{E}[\Delta T] = \mathbb{E}[T_{\text{find}}] + \mathbb{E}[T_{\text{cross}}].$$

This decomposition separates two different mechanisms. The first term measures how long it takes the process to reach the inter-community interface. The second term measures how long it takes to cross this interface once at least one bridge node is active.

Proposition. Exact transition time for a single bridge node. If $k = 1$, then, under uniform initial seeding in the clique, the expected time needed to find the bridge node is

$$\mathbb{E}[T_{\text{find}}] = \frac{H_{n-1}}{\beta_{\text{in}} n},$$

where

$$H_{n-1} = \sum_{m=1}^{n-1} \frac{1}{m}$$

is the harmonic number.

After the unique bridge node becomes active, the next community sees one active inter-community edge with rate β_{out}. Other internal events in the source clique do not change this single active export rate in the SI setting. Hence,

$$\mathbb{E}[T_{\text{cross}}] = \frac{1}{\beta_{\text{out}}}.$$

Therefore, for one bridge node, the expected time of one exact Micro transition in the first-passage baseline is

$$\mathbb{E}[\Delta T] = \frac{H_{n-1}}{\beta_{\text{in}} n} + \frac{1}{\beta_{\text{out}}}.$$

For a homogeneous chain, this gives the baseline expression

$$\mathbb{E}[T_{\text{last}}] = (L-1)\left(\frac{H_{n-1}}{\beta_{\text{in}} n} + \frac{1}{\beta_{\text{out}}}\right).$$

This formula shows that the expected arrival time in the last community is composed of identical local blocks. In each clique, the process first has to find the bridge node and then cross one active inter-community edge.

3.6.3 Several bridge nodes

For $k > 1$, the first phase has the same interpretation, but the bridge set contains several nodes.

Proposition. Expected bridge-finding time for a bridge set. Under the uniform seeding assumption inside each newly activated clique, the expected time until the first activation of the bridge set is

$$\mathbb{E}[T_{\text{find}}] = \sum_{m=1}^{n-k} \frac{\binom{n-k}{m}}{\binom{n}{m}} \cdot \frac{1}{\beta_{\text{in}} m(n-m)}.$$

Here m is the number of active nodes in the clique. The factor

$$\frac{\binom{n-k}{m}}{\binom{n}{m}}$$

is the probability that among the first m active nodes there is still no bridge node, while

$$\frac{1}{\beta m(n-m)}$$

is the expected residence time of the clique in the state with m active nodes, under SI dynamics with within-clique rate β_{in}.

The second phase is no longer identical to the single-bridge case. After the first bridge node becomes active, the export hazard is initially β. If additional bridge nodes become active before the export event occurs, the export hazard increases:

$$a_{\text{export}}^{\text{Micro}}(t) = \beta_{\text{out}} B_I(t),$$

where $B_I(t)$ is the number of active bridge nodes in the current community.

Proposition. Crossing time for several bridge nodes is bounded by the single-edge and fully active interface cases. For $k > 1$, since the post-activation export hazard lies between β_{out} and $\beta_{\text{out}}k$, the expected crossing time satisfies

$$\frac{1}{\beta_{\text{out}}k} \leq \mathbb{E}[T_{\text{cross}}] \leq \frac{1}{\beta_{\text{out}}}.$$

The lower bound corresponds to the case where all k bridge nodes are already active and the export hazard is $\beta_{\text{out}}k$. The upper bound corresponds to the case where only the first bridge node contributes to the export clock.

Hence, for $k > 1$, the expected time of one transition lies in the interval

$$\mathbb{E}[T_{\text{find}}] + \frac{1}{\beta_{\text{out}}k} \leq \mathbb{E}[\Delta T] \leq \mathbb{E}[T_{\text{find}}] + \frac{1}{\beta_{\text{out}}}.$$

Consequently, the expected arrival time in the last community satisfies

$$(L-1)\left(\mathbb{E}[T_{\text{find}}] + \frac{1}{\beta_{\text{out}}k}\right) \leq \mathbb{E}[T_{\text{last}}] \leq (L-1)\left(\mathbb{E}[T_{\text{find}}] + \frac{1}{\beta_{\text{out}}}\right).$$

This analytical interval is sufficient for the main benchmark interpretation. It shows how the parameters L, n, k, and β affect the expected propagation time without introducing a full finite-state recursion.

3.6.4 Relation to the MicroMacro approximation

In the MicroMacro model, the identities of the bridge nodes are not tracked explicitly. Instead, the macro layer observes only the aggregated contagiousness

$$\bar{i}_i(t) = \frac{i_i(t)}{N_i}.$$

For the transition

$$G_i \to G_{i+1},$$

the macro hazard has the form

$$\lambda_{i,i+1}^{\text{Macro}}(t) = \beta_{\text{out}} W_{i,i+1} \bar{i}_i(t) \bar{s}_{i+1}(t).$$

Before the next community is activated, its susceptible fraction is approximately

$$\bar{s}_{i+1}(t) \approx 1.$$

If the macro-link weight corresponds to the number of bridge contacts,

$$W_{i,i+1} = k,$$

then

$$\lambda_{i,i+1}^{\text{Macro}}(t) = \beta_{\text{out}} k \frac{i_i(t)}{n}.$$

Thus, the MicroMacro model replaces the exact bridge-dependent hazard

$$\beta_{\text{out}} B_I(t)$$

by the aggregated mean-field estimate

$$\beta k \frac{i_i(t)}{n}.$$

Proposition. A local timing shift accumulates along a homogeneous chain. Suppose that, for each transition in the homogeneous first-passage baseline, the Micro-Macro approximation induces the same expected timing shift

$$\delta = \mathbb{E}[\Delta T^{\mathrm{MicroMacro}}] - \mathbb{E}[\Delta T^{\mathrm{Micro}}].$$

Then the expected shift in the arrival time of the last community is

$$\mathbb{E}[T_{\mathrm{last}}^{\mathrm{MicroMacro}}] - \mathbb{E}[T_{\mathrm{last}}^{\mathrm{Micro}}] = (L-1)\delta.$$

This proposition follows directly from the decomposition of T_{last} into local delays. It explains why a chain benchmark is more sensitive than a single-transition benchmark: even a small local discrepancy can become visible after several consecutive transitions.

The clique–bridge–leaf benchmark showed that the mean-field replacement may produce premature leakage: the macro hazard can start accumulating before the exact Micro process has reached the bridge nodes. In a chain of cliques, the same local mechanism is repeated at several consecutive transitions. At the same time, the empirical timing shift of an implemented MicroMacro simulator may also reflect layer synchronization, macro-rate calibration, the import placement rule, and the numerical approximation of the hazard integral. For this reason, the sign and magnitude of the observed shift are reported through arrival-time distributions in the numerical benchmark.

3.6.5 Calibrated chain-of-cliques benchmark

We use the chain construction as a controlled simulator diagnostic. Complete graphs minimize within-community heterogeneity, so the benchmark focuses on repeated inter-community transmission. We compare the two implemented simulators through arrival timing, propagation-wave dynamics, accumulation of timing error, and sensitivity to interface width k, chain length L, and community size n. Runtime is reported only as an implementation diagnostic.

Before the main runs, we calibrated the transmission rate on the baseline chain $L = 10, n = 100, k = 5$. A pilot grid $\beta \in \{0.00075, 0.001, 0.0015, 0.002, 0.003, 0.005\}$, with 20 paired replications per value, showed that $\beta = 0.001$ places the terminal arrival distribution inside the time window without making the outbreak finish too early. The calibrated benchmark therefore uses

$$\beta = 0.001, \qquad \gamma = 0, \qquad \tau_{\mathrm{micro}} = 1, \qquad \beta_{\mathrm{macro}} = \beta,$$

with 100 paired replications per scenario. Within a replication, both simulators use the same generated network, initial node, and nominal run seed; because their event sequences differ, this pairing controls network and initialization variability but is not an event-by-event common-random-number coupling. Networks are regenerated across replications. In clique chains, each community is complete and k distinct inter-community node pairs are sampled uniformly for every adjacent pair of communities. Full time series are saved for every run. For trajectory plots, the solid curve is the pointwise median across replications and the shaded band is the interquartile range.

For community i, $W_1(T_i)$ is the Wasserstein-1 distance between the empirical Micro and MicroMacro arrival-time distributions. We also report $\mathrm{rel}\, W_1(T_i) = W_1(T_i)/\mathbb{E}[T_i^{\mathrm{Micro}}]$, which makes discrepancies comparable across time scales. These metrics use completed observations and are accompanied by completion probabilities. To include censored runs, we additionally report restricted mean survival time $\mathrm{RMST} = \mathbb{E}[\min(T_{\mathrm{last}}, T_{\mathrm{end}})]$, its MicroMacro-minus-Micro shift, and paired-bootstrap 95% confidence intervals based on

Sweep	Setting	Completion	W_1	rel. W_1	$\Delta q_{0.5}$	ΔRMST [95% CI]
interface	$k = 1$	1/100 vs 0/100	–	–	–	1.3 [0.0, 3.8]
interface	$k = 5$	68/100 vs 82/100	67.050	0.035	70.768	-75.6 [-197.3, 34.8]
interface	$k = 10$	100/100 vs 100/100	130.062	0.098	-68.878	-127.9 [-217.8, -39.7]
interface	$k = 20$	100/100 vs 100/100	140.840	0.160	-118.846	-140.8 [-190.6, -89.4]
length	$L = 5$	88/100 vs 89/100	61.925	0.071	35.364	20.2 [-63.8, 110.2]
length	$L = 10$	68/100 vs 82/100	67.050	0.035	70.768	-75.6 [-197.3, 34.8]
length	$L = 15$	75/100 vs 86/100	183.057	0.059	-165.430	-248.4 [-428.7, -51.2]
size	$n = 50$	59/100 vs 65/100	224.902	0.108	-261.604	-168.9 [-266.4, -69.7]
size	$n = 100$	68/100 vs 82/100	67.050	0.035	70.768	-75.6 [-197.3, 34.8]
size	$n = 200$	79/100 vs 77/100	105.121	0.057	-171.341	-67.3 [-178.2, 49.3]

Table 1: Calibrated chain–of–cliques benchmark. Completion is Micro versus MicroMacro by T_{end}. W_1 and relative W_1 use completed terminal arrivals. $\Delta\text{RMST} = \text{RMST}_{\text{MicroMacro}} - \text{RMST}_{\text{Micro}}$ uses all runs, with paired-bootstrap 95% confidence intervals.

1000 resamples. Mean and quantile shifts are also MicroMacro minus Micro. The paired cumulative error $T_i^{\text{MicroMacro}} - T_i^{\text{Micro}}$ compares matched replications from the simulation start and is summarized by its median and interquartile range.

Baseline scenario. The baseline setting is $L = 10$, $n = 100$, $k = 5$, and $T_{\text{end}} = 2500$. The calibrated rate produces propagation waves that occupy most of the time horizon rather than ending near the beginning of the plot. Micro reached the last community in 68 replications, and MicroMacro in 82. Conditional on completed terminal arrivals, $W_1(T_{\text{last}}) = 67.050$, or 3.5% of the Micro mean terminal-arrival time; its bootstrap 95% interval is $[2.6\%, 9.2\%]$. The completed-sample median shift is 70.768, but the censoring-aware RMST shift is -75.6 with 95% CI $[-197.3, 34.8]$. Thus the baseline shows a modest distributional difference but does not establish a systematic early-or-late bias.

Interface-width sensitivity. The first sweep varies $k \in \{1, 5, 10, 20\}$ while keeping $L = 10$, $n = 100$, and $T_{\text{end}} = 2500$. The narrow-interface case $k = 1$ almost never reaches the last community by this horizon, so terminal W_1 is not meaningful there. This is itself informative: the interface is too narrow for reliable terminal comparison at this horizon. At $k = 5$, the completed terminal distributions are close in relative terms. At $k = 10$ and $k = 20$, all runs complete, with relative $W_1(T_{\text{last}}) = 0.098$ and 0.160, respectively. The corresponding negative median shifts indicate earlier terminal arrivals in MicroMacro for the wider interfaces.

Chain-length sensitivity. The second sweep varies $L \in \{5, 10, 15\}$ with $n = 100$ and $k = 5$, using $T_{\text{end}} = 1500, 2500, 4000$, respectively. Absolute $W_1(T_{\text{last}})$ increases from 61.925 at $L = 5$ to 183.057 at $L = 15$, while relative W_1 stays in the range 0.035–0.071. Thus the absolute timing discrepancy grows with chain length, but part of this growth is due to the longer time scale. The signed shift also changes across settings, so the benchmark does not support a single uniform early-or-late bias.

Community-size sensitivity. The third sweep varies $n \in \{50, 100, 200\}$ with $L = 10$ and $k = 5$. The smallest communities give the largest relative discrepancy: 0.108 for $n = 50$, compared with 0.035 for $n = 100$ and 0.057 for $n = 200$. The $n = 50$ case also

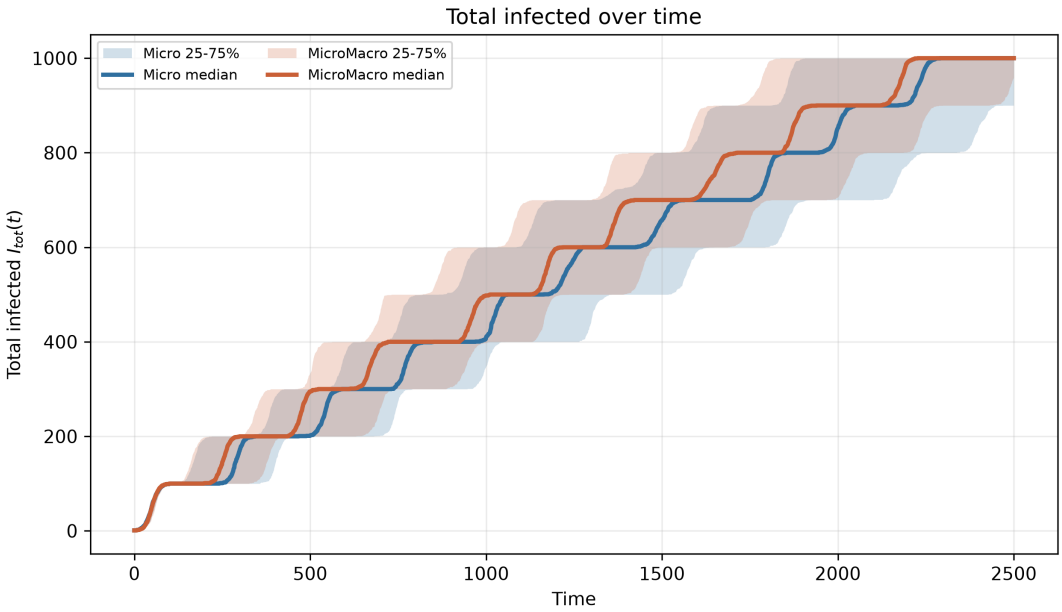


Figure 2: Baseline total infected count $I_{\mathrm{tot}}(t)$. Solid curves are pointwise medians over 100 replications; shaded regions are interquartile ranges.

has lower terminal completion. This suggests that smaller communities amplify stochastic timing differences and make the aggregate interface approximation less stable.

Runtime. The current implementation does not show a runtime advantage for MicroMacro on these moderate chain benchmarks. Across the calibrated scenarios, the runtime ratio $\mathrm{runtime}_{\mathrm{Micro}}/\mathrm{runtime}_{\mathrm{MicroMacro}}$ remains below 1, ranging from 0.057 to 0.450. Thus MicroMacro is slower than the full Micro simulator in the present Python implementation and parameter range. This should not be interpreted as an asymptotic complexity statement, but it is important for evaluating the current code.

Interpretation. The calibrated chain benchmark shows that the two simulators are not identical, but the size and direction of the difference depend on the regime. In the baseline, terminal distributions are close relative to the time scale, but completion differs and paired arrival errors have wide interquartile bands. Narrow interfaces mostly create censoring at the chosen horizon, wider interfaces can make MicroMacro arrive earlier, longer chains increase absolute timing error, and smaller communities amplify relative discrepancy. The benchmark should therefore be read as a full-system comparison of macro-rate abstraction, import placement, synchronization, and the choice $\beta_{\mathrm{macro}} = \beta$, not as an isolated test of a single mechanism.

3.7 Chain of sparse connected communities benchmark

The chain–of–cliques benchmark intentionally removes most within-community heterogeneity: once a community is seeded, the complete graph mixes quickly. We therefore add a second chain benchmark in which every community is an independently generated connected Erdos–Renyi graph. This benchmark is more demanding for the MicroMacro approximation because the community-level infected fraction is a weaker proxy for the infection state of the boundary nodes. We refer to this construction as the chain of sparse connected communities.

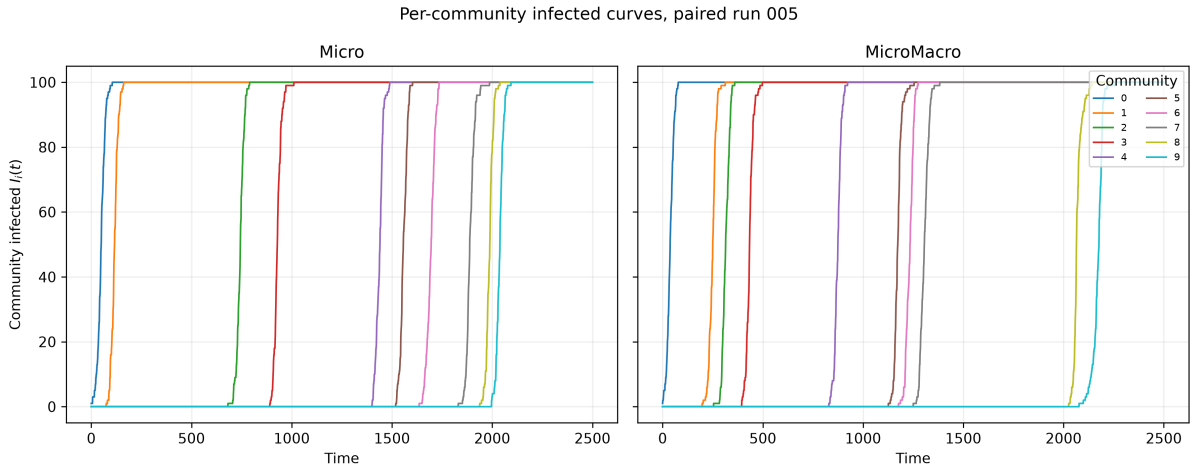


Figure 3: Baseline per-community infected curves for one paired completed replication. This figure illustrates the sequential propagation mechanism; it is not an average over replications.

Scenario	Micro comp.	MM comp.	$E[T^M_{\text{last}}]$	$E[T^{MM}_{\text{last}}]$	Rel. W_1	Mean shift	ΔRMST [95% CI]
baseline $k = 5, d = 6$	0.99	1.00	2858.0	1640.5	0.426	-1217.5	-1228.9 [-1379.4, -1089.6]
$k = 1, d = 6$	0.02	0.19	3937.7	2778.9	0.294	-1158.8	-230.8 [-366.0, -125.3]
$k = 10, d = 6$	1.00	1.00	2138.3	1078.3	0.496	-1060.0	-1060.0 [-1181.5, -951.2]
$k = 20, d = 6$	1.00	1.00	1608.8	851.8	0.471	-757.1	-757.1 [-846.0, -663.1]
$k = 5, d = 4$	0.43	0.98	3556.4	2126.7	0.402	-1429.6	-1645.1 [-1822.4, -1455.8]
$k = 5, d = 10$	1.00	1.00	1976.9	1392.6	0.296	-584.3	-584.3 [-696.2, -482.1]

Table 2: Chain of sparse connected communities benchmark. M denotes Micro and MM denotes MicroMacro. Means, mean shifts, and relative W_1 use completed terminal arrivals. ΔRMST uses all runs and paired-bootstrap 95% confidence intervals.

All scenarios use a chain of $L = 10$ communities with $n = 100$ nodes per community. The baseline internal mean degree is approximately $d = 6$, implemented as $p = d/(n - 1) = 0.0606$. Inter-community transmission uses k explicit edges between neighboring communities. The SI setting is used with

$$\beta = 0.003, \qquad \gamma = 0, \qquad \tau_{\text{micro}} = 1, \qquad \beta_{\text{macro}} = \beta, \qquad T_{\text{end}} = 4000.$$

Each scenario uses 100 paired replications under the same pairing convention as the clique benchmark. Internally, a graph with exactly $m = \text{round}(pn(n-1)/2)$ edges is sampled uniformly and resampled until connected. For each adjacent community pair, k distinct node pairs are selected uniformly as explicit inter-community edges. A new network is generated for every replication and shared by Micro and MicroMacro. Full trajectories are stored for every run. Completed-only metrics are reported together with completion, while RMST incorporates all censored runs. The interface sweep changes only k, and the density sweep changes only d, so comparisons use a common horizon and censoring rule.

Figure 17 shows that the discrepancy accumulates along the sparse chain rather than appearing only at the terminal community. Relative $W_1(T_i)$ rises from about 0.08 at the first transition to 0.426 at the final community, while the increasingly negative mean and quantile shifts show systematic premature propagation in MicroMacro. Because baseline terminal completion is 0.99 for Micro and 1.00 for MicroMacro, this trend is only weakly affected by terminal censoring.

The sparse connected benchmark produces a qualitatively different conclusion from the clique benchmark. In the baseline completed sample, Micro reaches the final community at mean time 2858.0, whereas MicroMacro reaches it at 1640.5. The terminal relative W_1

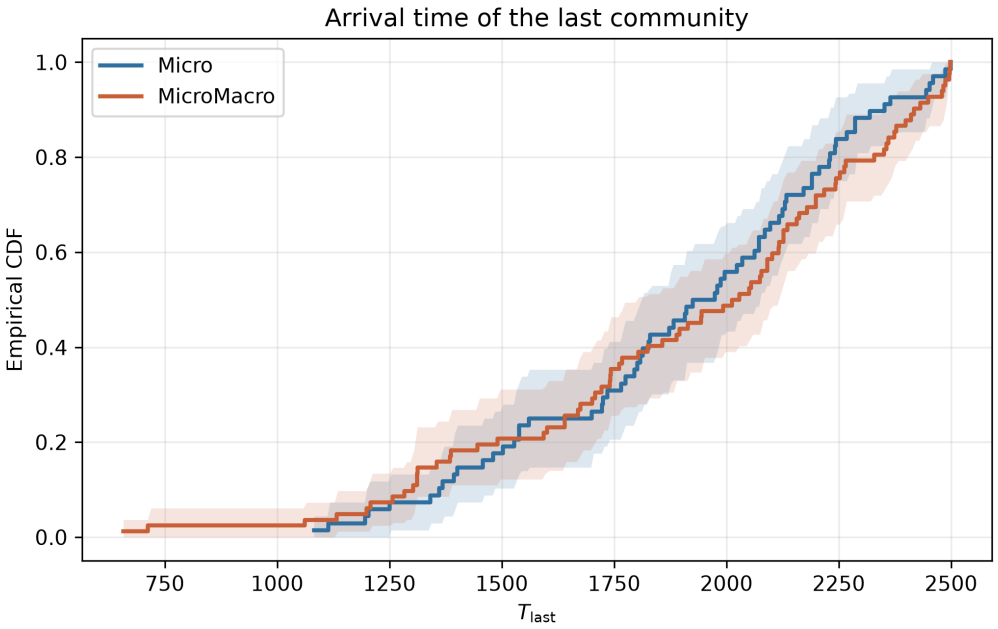


Figure 4: Baseline empirical CDF of T_{last} for completed terminal arrivals in the $L = 10$, $n = 100$, $k = 5$ chain. Shaded regions are bootstrap 95% confidence bands for the empirical CDFs; completion probabilities are reported separately in Table 1.

is 0.426, with bootstrap 95% CI $[0.381, 0.470]$. More importantly, the all-run RMST shift is -1228.9, with CI $[-1379.4, -1089.6]$. Both the completed-only and censoring-aware analyses therefore show substantially earlier downstream arrival in MicroMacro.

This behavior is consistent with the mechanism isolated by the clique–bridge–leaf benchmark. In sparse random connected communities, the aggregate infected fraction can grow before the boundary nodes responsible for export have been infected. The macro layer therefore accumulates hazard from a community average, while the full Micro process waits for infection to reach the sampled interface nodes. Wider interfaces reduce absolute terminal times but do not remove the discrepancy. For $k = 1$, completed-only W_1 is not comparable to fully observed scenarios; however, the RMST shift remains negative, -230.8 with CI $[-366.0, -125.3]$, confirming earlier MicroMacro propagation even when all censored runs are retained.

The internal-density sweep shows the expected direction. With $d = 4$, Micro completion falls to 0.43, while MicroMacro completes in 0.98 of runs, indicating that the macro abstraction strongly overestimates the ability of infection to reach export-relevant locations. With $d = 10$, the discrepancy is smaller but still substantial: relative $W_1 = 0.296$ and the mean terminal shift is -584.3. Thus increasing internal mixing improves agreement, but the sparse connected benchmark remains a clear stress test for the current MicroMacro implementation.

The runtime ratios are reported only as implementation diagnostics. They range from 0.037 to 0.068 in these moderate-size scenarios and do not constitute a general complexity or speedup claim.

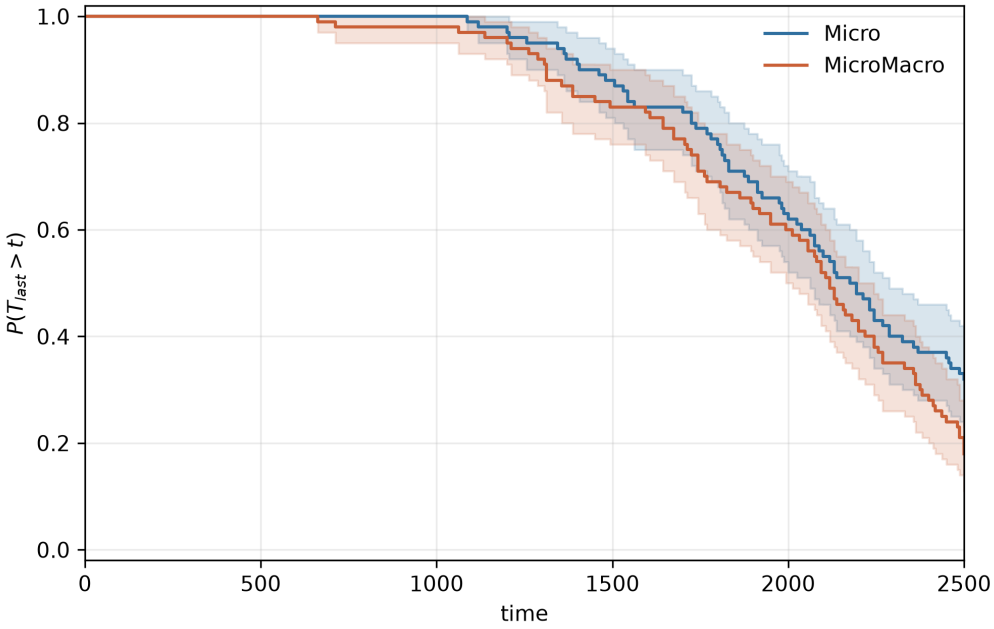


Figure 5: Censoring-aware survival curves $P(T_{\text{last}} > t)$ for the calibrated clique baseline. Shaded regions are paired-bootstrap 95% pointwise bands; runs without terminal arrival remain at risk until $T_{\text{end}} = 2500$.

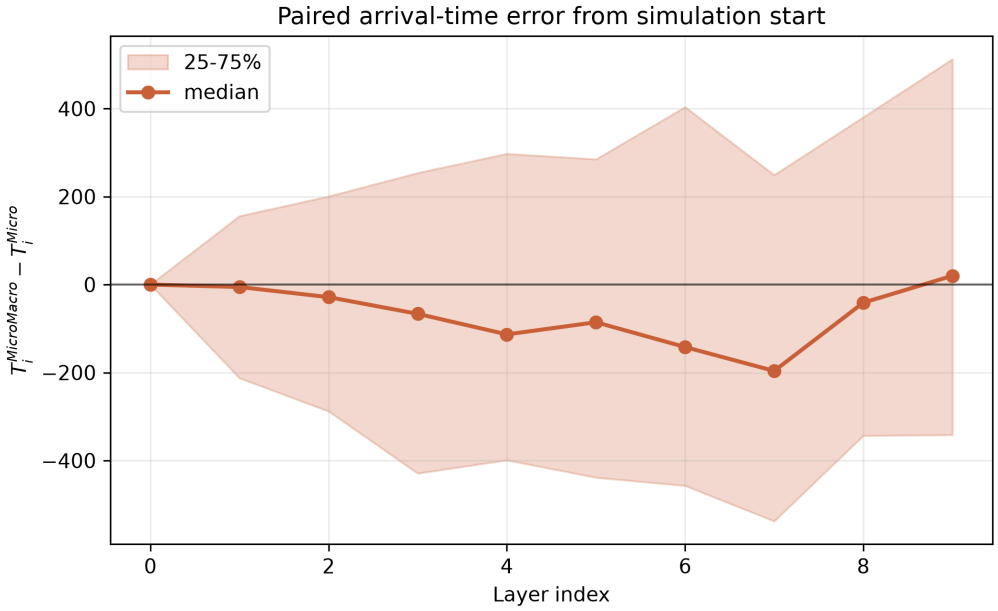


Figure 6: Baseline paired cumulative arrival-time error from the simulation start. The plotted quantity is $T_i^{\text{MicroMacro}} - T_i^{\text{Micro}}$, not the local delay $\Delta T_i = T_{i+1} - T_i$. The line is the median over paired observations and the band is the interquartile range.

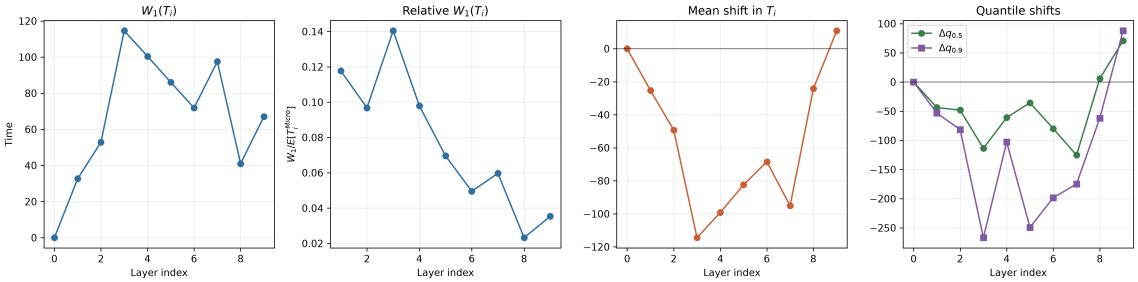


Figure 7: Baseline layer-wise distributional discrepancy. The panels show absolute $W_1(T_i)$, relative $W_1(T_i)$, mean shift, and quantile shifts across the chain.

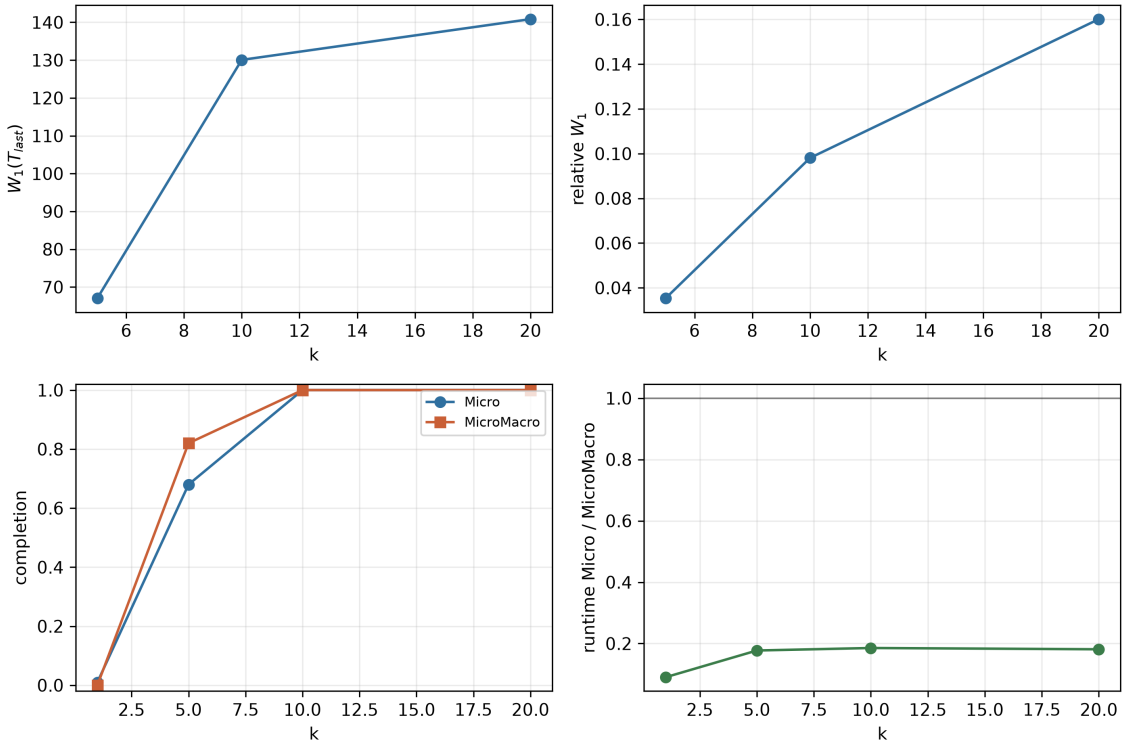


Figure 8: Interface-width sweep for $L = 10$, $n = 100$. Terminal distributional metrics are omitted where terminal completion is effectively absent.

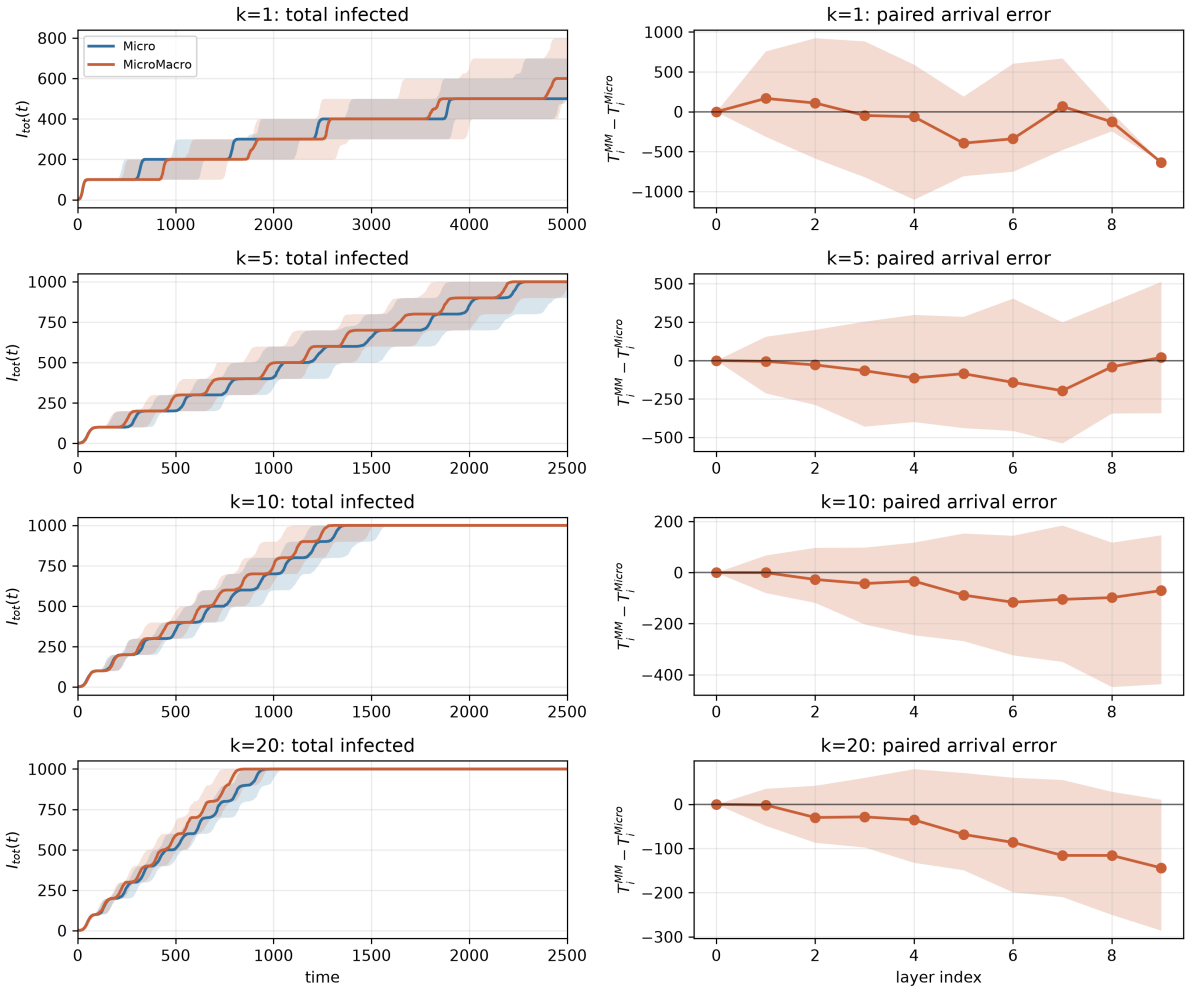


Figure 9: Interface-width dynamics for $k = 1, 5, 10, 20$. Left panels show median total infected curves with interquartile bands. Right panels show paired cumulative arrival-time errors $T_i^{\mathrm{MicroMacro}} - T_i^{\mathrm{Micro}}$, with median and interquartile band over paired observations. The $k = 1$ trajectory panel uses its separately simulated $T_{\mathrm{end}} = 5000$ horizon to reveal the slow propagation wave; the comparative metrics in Table 1 use the common $T_{\mathrm{end}} = 2500$ sweep and are not mixed with this extended-horizon visualization.

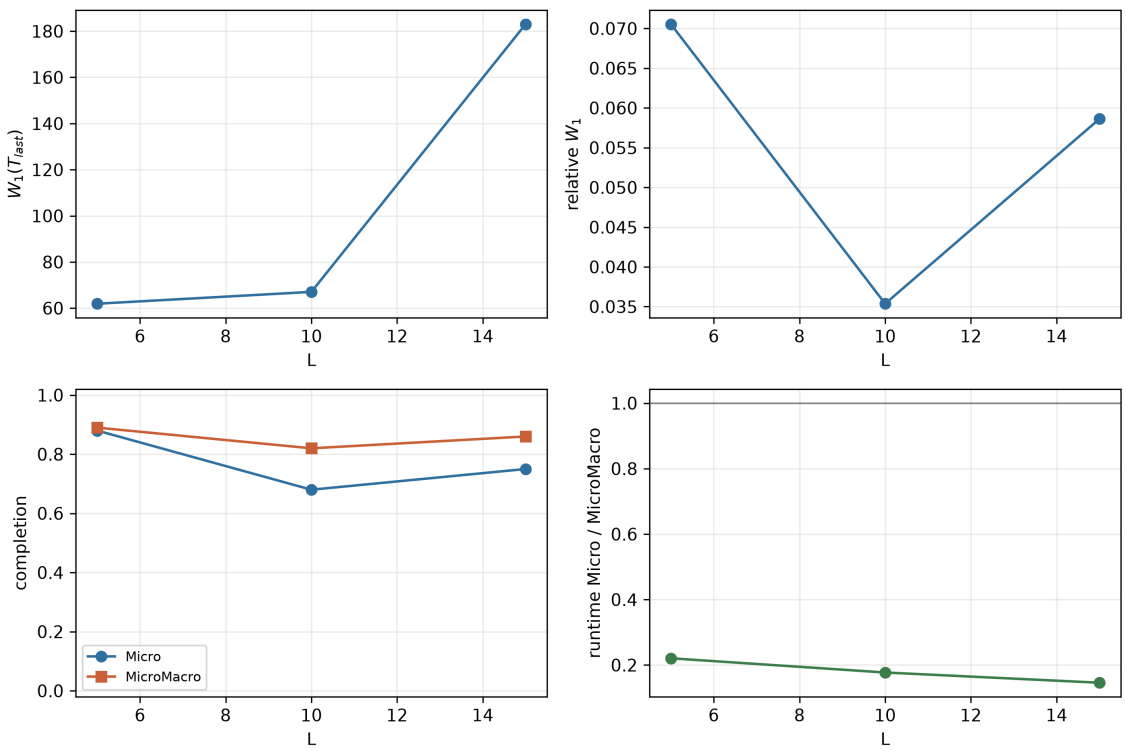


Figure 10: Chain-length sweep with $n = 100$, $k = 5$. Absolute terminal discrepancy increases with L, while relative discrepancy remains moderate.

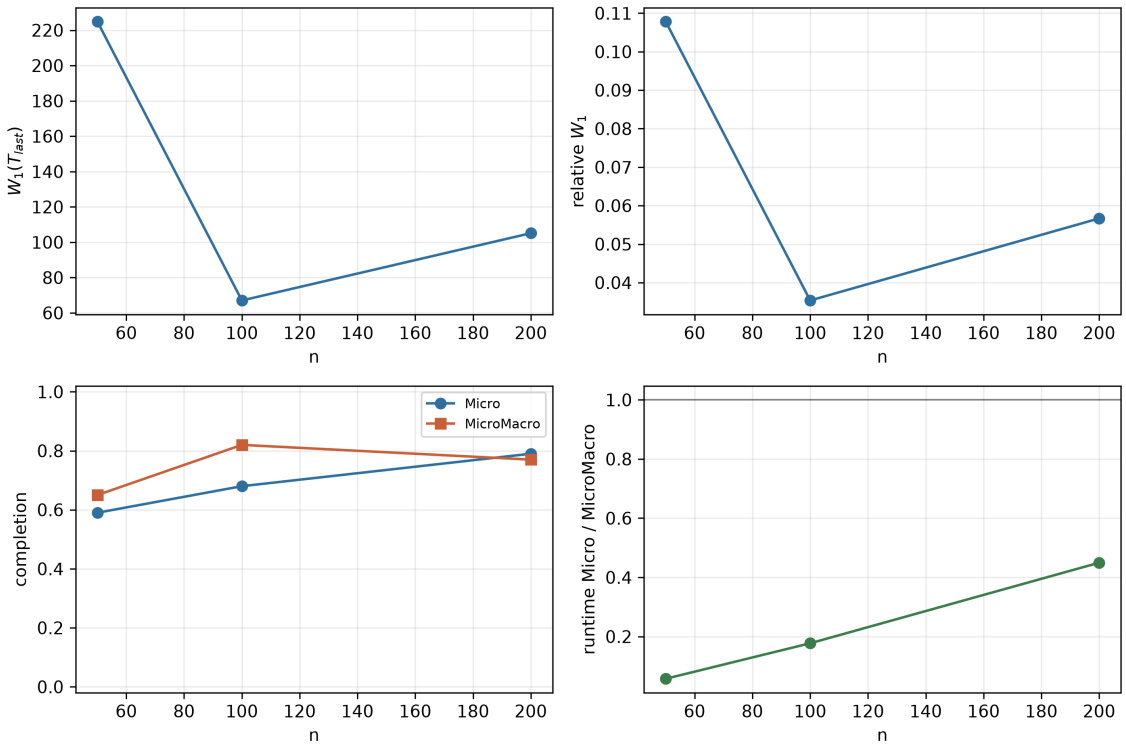


Figure 11: Community-size sweep with $L = 10$, $k = 5$. Smaller communities show larger relative terminal discrepancy and lower completion at the calibrated horizon.

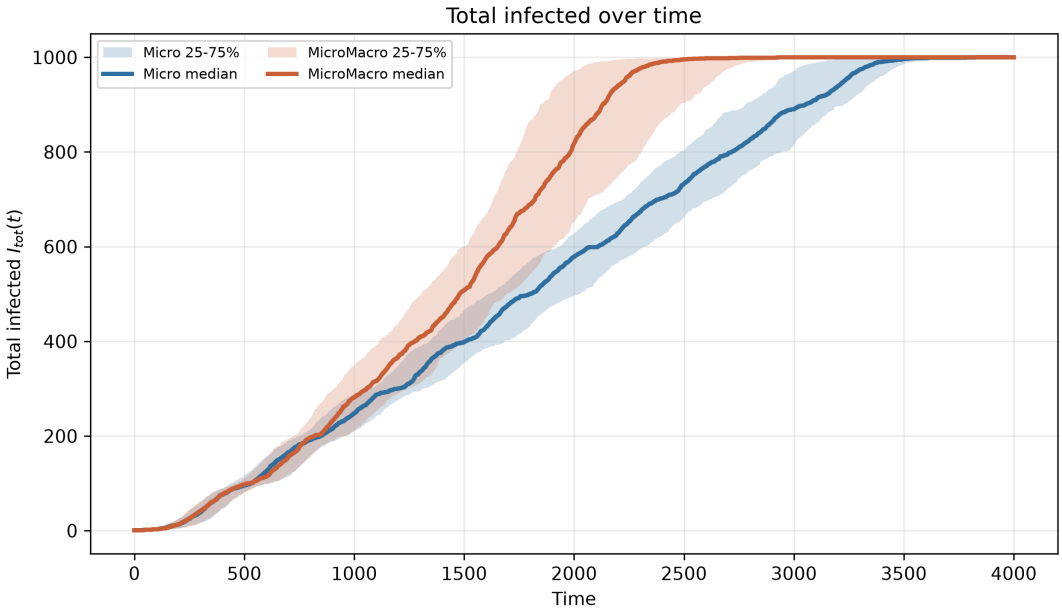


Figure 12: Baseline aggregate infected trajectories for the chain of sparse connected communities. Solid curves show pointwise medians over 100 paired replications; shaded bands show interquartile ranges.

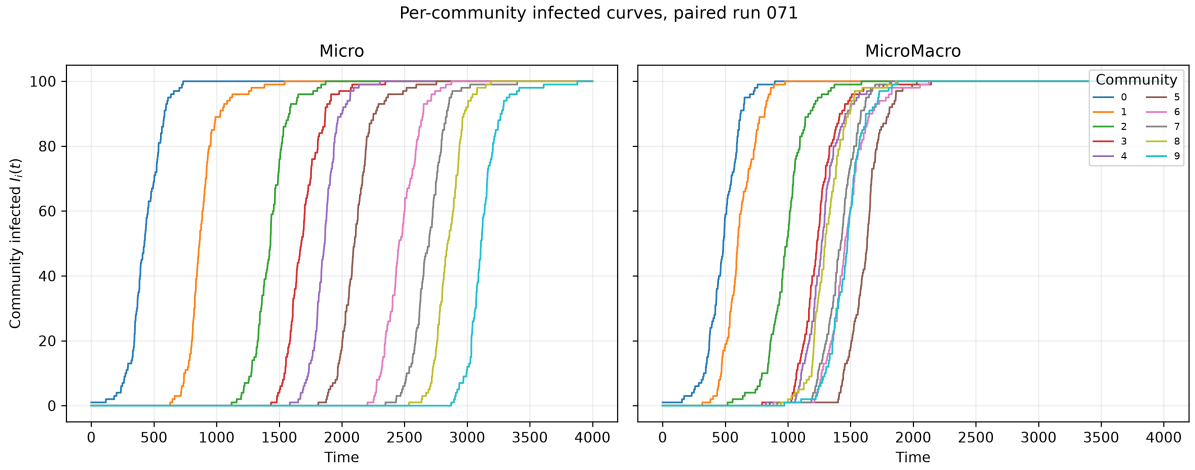


Figure 13: One paired baseline trajectory, shown by community. This plot illustrates the propagation mechanism only; distributional conclusions are based on all 100 replications.

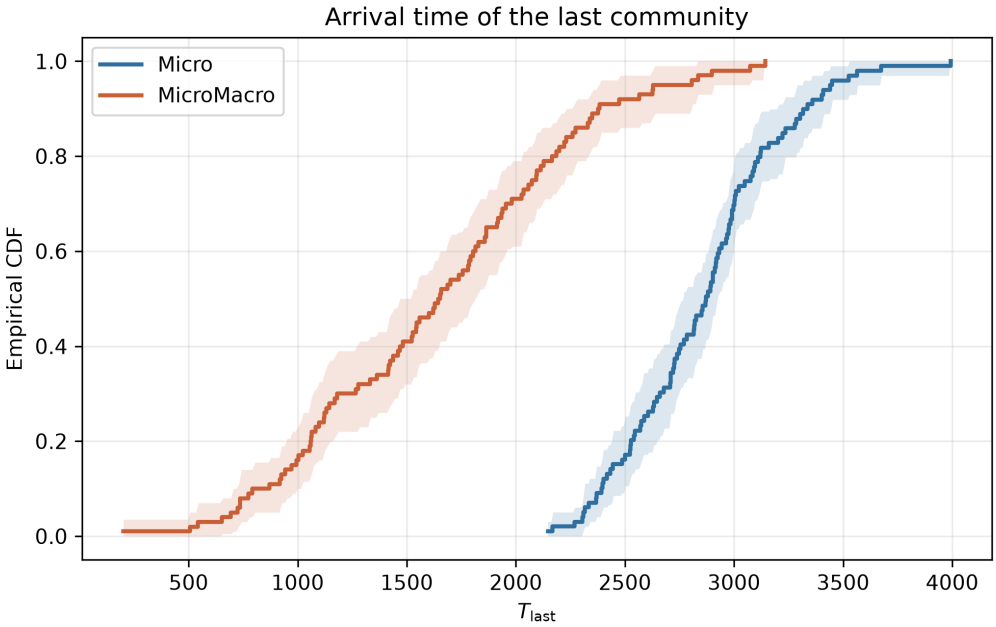


Figure 14: Baseline empirical CDF of T_{last} for completed terminal arrivals in the chain of sparse connected communities. Completion probabilities are reported separately in Table 2.

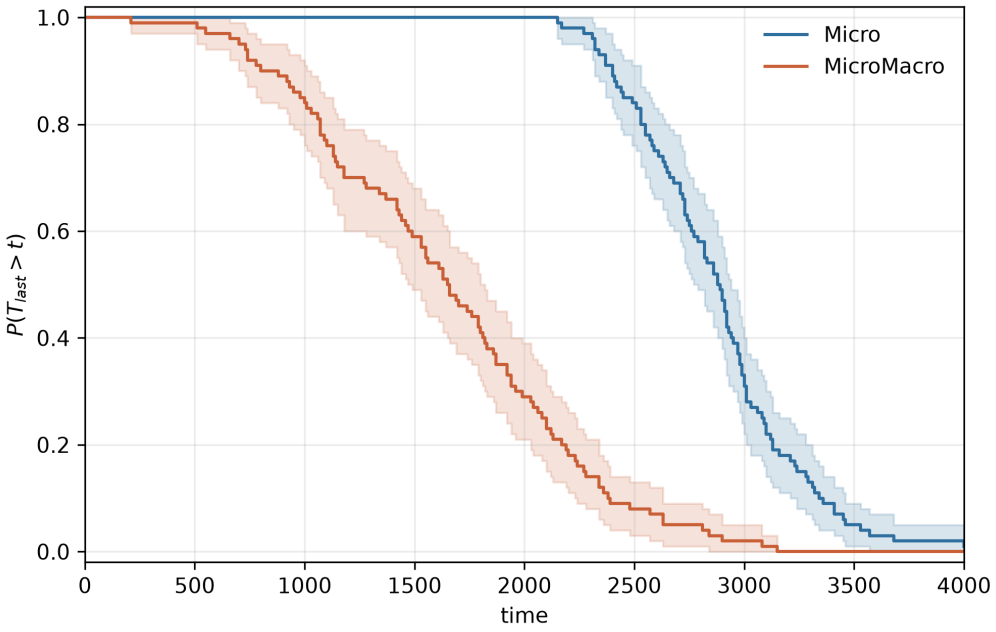


Figure 15: Censoring-aware survival curves $P(T_{\text{last}} > t)$ for the sparse connected baseline, with paired-bootstrap 95% pointwise bands.

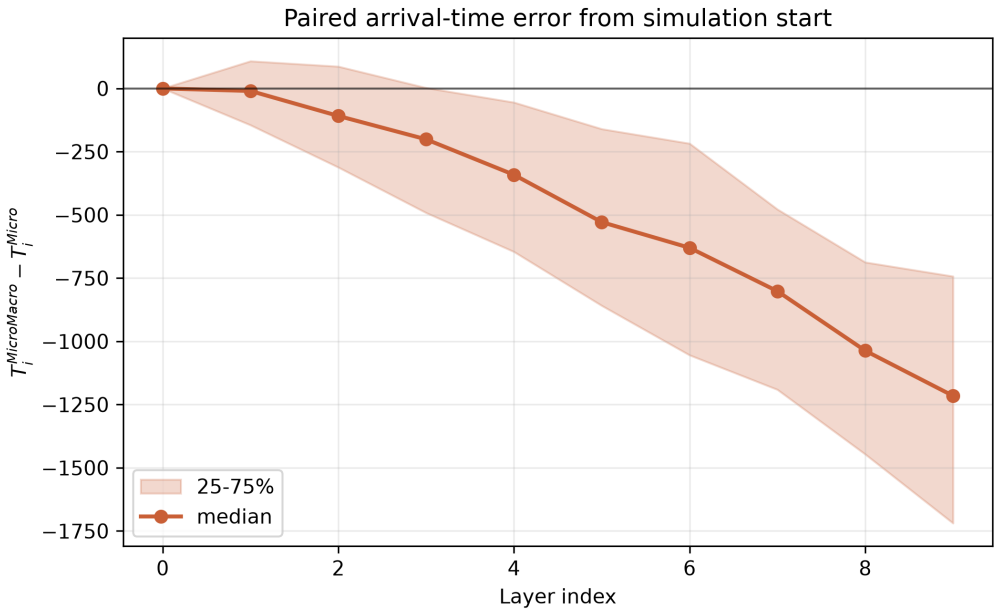


Figure 16: Baseline paired cumulative arrival-time error $T_i^{MM} - T_i^{M}$ across the chain. The solid curve is the median over paired observations and the band is the interquartile range.

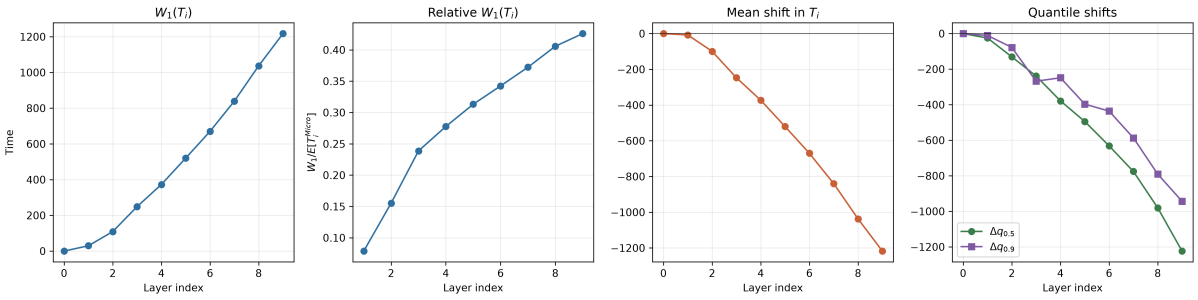


Figure 17: Layer-wise arrival-time discrepancy in the baseline sparse connected chain. Absolute and relative $W_1(T_i)$, mean shifts, and quantile shifts are computed from completed observations at each community; shifts are MicroMacro minus Micro.

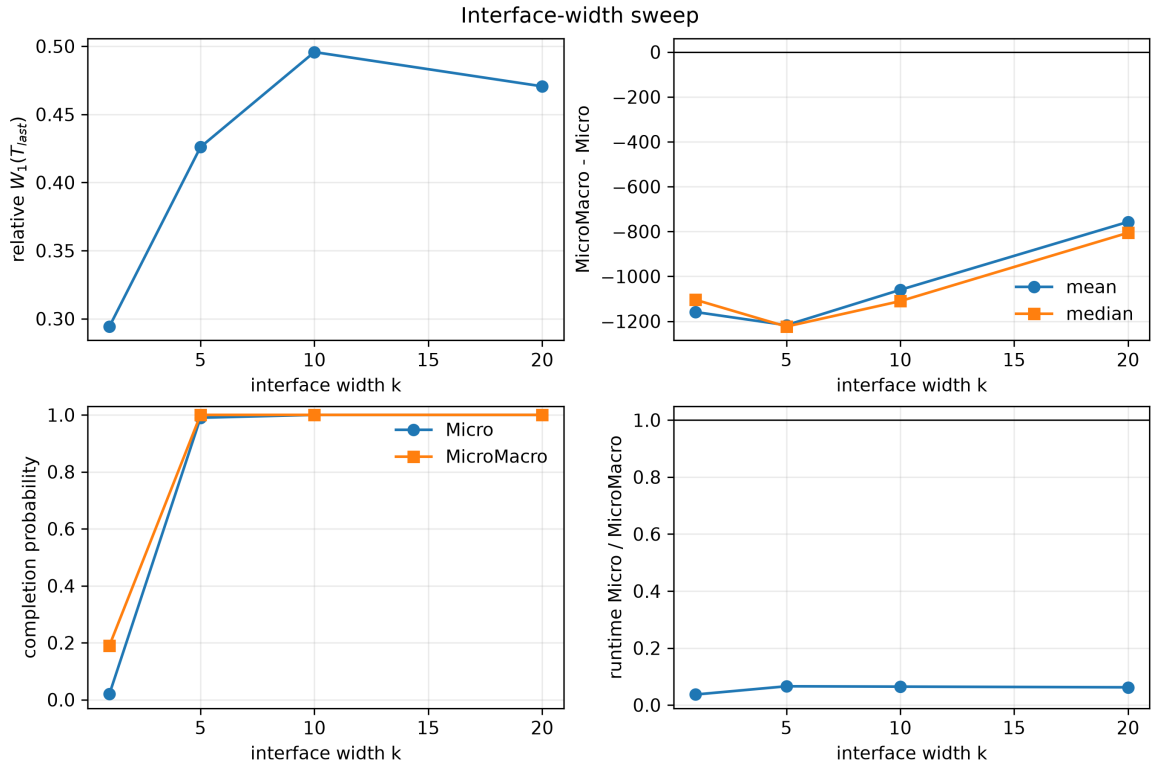


Figure 18: Interface-width sweep for the chain of sparse connected communities, with $d \approx 6$. Each panel changes only k, so differences are attributable to interface width at fixed community size, internal density, transmission rate, and time horizon.

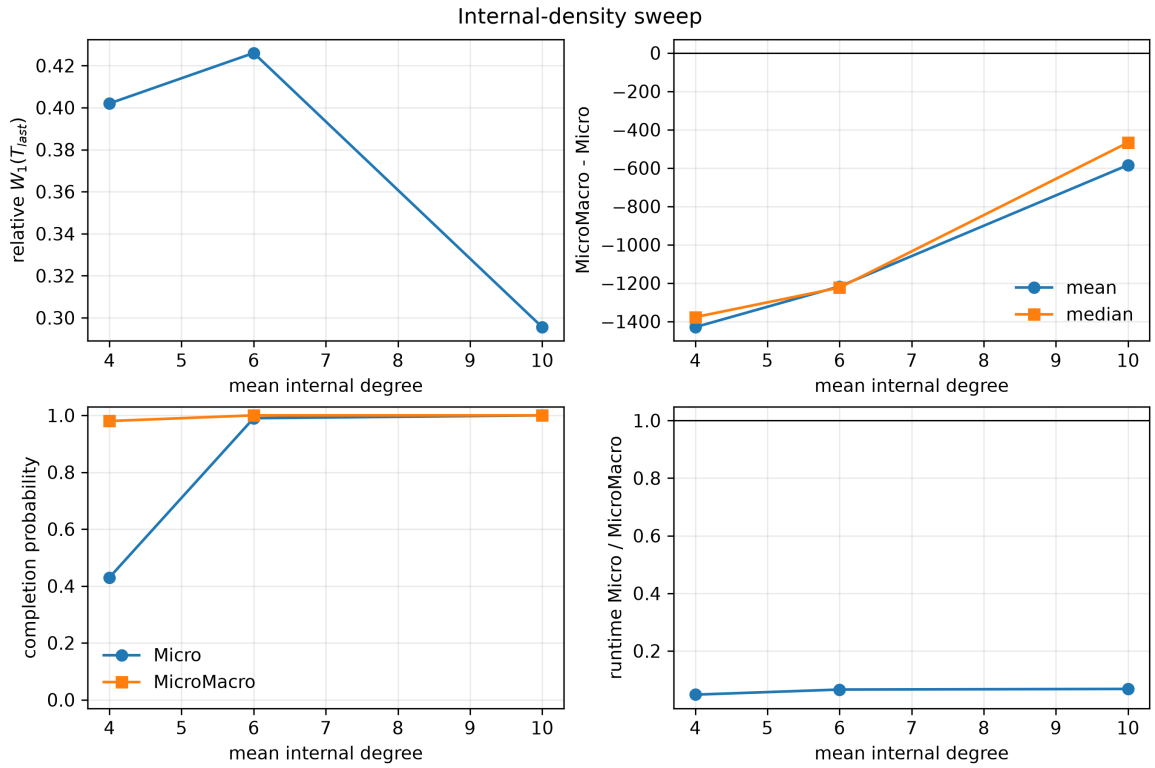


Figure 19: Internal-density sweep for the chain of sparse connected communities, with $k = 5$. Each panel changes only the expected internal degree d.

4 Conclusions and Outlook

This paper studied a two-scale continuous-time stochastic simulator for epidemic spreading on community-structured networks. The method preserves exact CTMC/Gillespie dynamics inside each community, while replacing the full collection of microscopic cross-community infection channels by macro-level stochastic import hazards. The main purpose of the analysis was not merely to define this simulator, but to identify precisely what is approximated when the inter-community layer is coarse-grained.

The key conclusion is that local exactness and global accuracy are distinct. Conditional on a fixed sequence of import times and target nodes, each micro engine evolves according to the correct local CTMC. Any discrepancy from the full-network reference therefore enters through the construction of the import process: the macro rate may not equal the exact boundary-edge intensity, the import-placement rule may not match the edge-conditioned target distribution, and numerical synchronization may introduce additional timing error. This decomposition gives a practical diagnostic framework for evaluating two-scale epidemic simulators.

The clique–bridge–leaf benchmark isolates the most important structural issue. In the full microscopic model, a receiving leaf can be infected only after the source infection reaches one of the bridge nodes. In the macro model, however, the import hazard may begin accumulating as soon as the source community has a positive infected fraction. Under exchangeability assumptions this macro hazard can be the conditional mean of the exact bridge hazard, but the corresponding arrival-time distribution can still be biased because survival probabilities depend on the exponential of accumulated hazard. Thus, matching instantaneous mean hazards is not sufficient for matching event-time laws.

The chain-of-cliques benchmark shows why this local interface error matters at the network scale. In complete communities, the calibrated baseline discrepancy is modest and its censoring-aware RMST confidence interval includes zero, although wider interfaces and longer chains produce detectable early-arrival shifts in several settings. In sparse connected communities the result is much stronger: the baseline relative Wasserstein discrepancy is about 43%, and the RMST shift is -1228.9 with a paired-bootstrap 95% interval $[-1379.4, -1089.6]$. The layer-wise errors increase along the chain, and increasing internal density reduces but does not eliminate the premature MicroMacro propagation. These numerical results are full-system approximation diagnostics; runtime measurements are implementation-specific and are not the principal claim.

The two-scale approximation appears most reliable when boundary nodes are representative of their communities and within-community mixing is fast relative to export. It is least reliable when infection status is correlated with boundary position or when the macro layer accumulates substantial hazard before the true interface nodes are infected. The sparse-community experiments show that widening the interface alone is not sufficient when aggregate prevalence remains a poor proxy for boundary exposure. In such regimes, the macro layer should be treated as an effective approximation rather than an exact coarse-graining of the microscopic network.

4.1 Outlook

Several extensions would make the framework more publication-ready and more useful computationally. First, the macro rate λ_{ij} should be calibrated against the exact edge-level intensity $a^{\star}_{ij}$ in benchmark networks where both are available. Second, import placement should be made bridge-aware, for example by using boundary-degree information or an approximation to the exact edge-conditioned distribution $\pi^{\star}_{ij}$. Third, the synchronization

step should be tested by varying the micro update interval and measuring the resulting hazard-integration error. Fourth, numerical comparisons should include censoring-aware statistics for arrival times, confidence intervals across replications, and larger networks in which the intended complexity advantage of the two-scale architecture can be evaluated fairly.

A natural next version of the method would therefore combine aggregate mobility weights with a small amount of retained boundary information. This would preserve the main computational advantage of the macro layer while reducing the dominant source of interface bias. From the modeling perspective, such a hybrid bridge-aware macro layer is also more realistic: large-scale mobility data determine how strongly communities are coupled, while boundary or gateway structure determines where imported infections can actually enter.

References

[Ajelli et al., 2010] Ajelli, M., Gonçalves, B., Balcan, D., Colizza, V., Hu, H., Ramasco, J. J., Merler, S., and Vespignani, A. (2010). Comparing large-scale computational approaches to epidemic modeling: Agent-based versus structured metapopulation models. *BMC Infectious Diseases*, 10:190.

[Anderson, 2007] Anderson, D. F. (2007). A modified next reaction method for simulating chemical systems with time dependent propensities and delays. *The Journal of Chemical Physics*, 127(21):214107.

[Balcan et al., 2010] Balcan, D., Colizza, V., Gonçalves, B., Hu, H., Ramasco, J. J., and Vespignani, A. (2010). Modeling the spatial spread of infectious diseases: The global epidemic and mobility computational model. *Journal of Computational Science*, 1(3):132–145.

[Barbosa et al., 2018] Barbosa, H., Barthelemy, M., Ghoshal, G., James, C. R., Lenormand, M., Louail, T., Menezes, R., Ramasco, J. J., Simini, F., and Tomasini, M. (2018). Human mobility: Models and applications. *Physics Reports*, 734:1–74.

[Colizza et al., 2007] Colizza, V., Pastor-Satorras, R., and Vespignani, A. (2007). Reaction–diffusion processes and metapopulation models in heterogeneous networks. *Nature Physics*, 3(4):276–282.

[Colizza and Vespignani, 2008] Colizza, V. and Vespignani, A. (2008). Epidemic modeling in metapopulation systems with heterogeneous coupling pattern: Theory and simulations. *Journal of Theoretical Biology*, 251(3):450–467.

[Gillespie, 1977] Gillespie, D. T. (1977). Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry*, 81(25):2340–2361.

[Kerr et al., 2021] Kerr, C. C., Stuart, R. M., Mistry, D., Abeysuriya, R. G., Rosenfeld, K., Hart, G. R., Núñez, R. C., Cohen, J. A., Selvaraj, P., Hagedorn, B., George, L., Jastrzębski, M., Izzo, A. S., Fowler, G., Palmer, A., Delport, D., Scott, N., Kelly, S. L., Bennette, C. S., Wagner, B. G., Chang, S. T., Oron, A. P., Wenger, E. A., Panovska-Griffiths, J., Famulare, M., and Klein, D. J. (2021). Covasim: An agent-based model of covid-19 dynamics and interventions. *PLoS Computational Biology*, 17(7):e1009149.

[Kuryliak et al., 2025] Kuryliak, Y., Emmerich, M. T., and Dosyn, D. (2025). Simulating epidemic peak dynamics on complex networks using efficient gillespie algorithms. *Infection, Genetics and Evolution*, 132:105768.

[Vestergaard and Génois, 2015] Vestergaard, C. L. and Génois, M. (2015). Temporal gillespie algorithm: Fast simulation of contagion processes on time-varying networks. *PLoS Computational Biology*, 11(10):e1004579.